From Zero to Propagators

Orders, Lattices, and Propagator Networks in Lean 4

> *"The propagator model is to functional programming*
> *what constraint logic programming is to logic programming:*
> *a radical generalisation that turns programs into networks*
> *of co-operating, partial knowledge sources."*
>
> — Inspired by Radul & Sussman (2009)

A Self-Contained Journey Through Order Theory
to Verified Constraint Propagation

Lean 4 code requires no external libraries beyond the standard prelude.

Edition 1.0

Preface

This book has a single grand ambition: to take you from the very first principles of ordered structures all the way to building a working *propagator network* — a powerful computational model that arises naturally from the mathematics of lattices — and to do it with complete rigour in Lean 4.

Who this book is for. We assume you are comfortable reading mathematics at the level of a second-year undergraduate: sets, functions, logical quantifiers, and basic proof techniques are fair game. No prior knowledge of abstract algebra or Lean is required, though some familiarity with functional programming will help when reading the longer code sections.

The Lean 4 philosophy here. Every definition and theorem in this book has a corresponding Lean 4 encoding. We deliberately avoid Mathlib to keep the dependency footprint at zero and to force you to understand every axiom rather than importing it from a library. Where a Mathlib equivalent exists we point it out in a margin note.

How to read this book. Part I builds the algebraic foundations: relations, partial orders, and lattices. Part II shows how these structures give rise to a beautiful computational model — the propagator network. Part III contains two capstone projects that demonstrate the full power of the model: a Sudoku solver driven entirely by constraint propagation, and a type-inference engine for a small lambda calculus. By the time you reach the end of Part III the connection between the mathematics and the code should feel inevitable.

A note on Lean Unicode. All code listings use the Unicode characters that Lean 4 accepts natively: `∀ ∃ → ← ↔ ≤ ≥ ⊓ ⊔ ⊥ ⊤ ∈ ∉ α β γ ×` and so on. When reading the code, treat these as single characters exactly as Lean treats them. In Lean 4 you enter them with backslash sequences (e.g. `\le` for `≤`); your editor should substitute them live.

Contents

Part I

The Algebra of Order

Chapter 1

Relations and Partial Orders

> *"The concept of an ordering is one of the most basic in all of mathematics, and yet we rarely pause to examine what it actually requires."*

Before we can study lattices or build propagator networks, we need a precise vocabulary for talking about *order*: what it means for one thing to be "at most" another, how such a concept can be captured in a type, and which algebraic laws make it useful.

1.1 Binary Relations

Definition — Binary Relation

A *binary relation* on a set A is a subset $R \subseteq A \times A$. We write $a\ R\ b$ (or $R(a, b)$) as shorthand for $(a, b) \in R$.

Lean 4 encodes a relation as a function returning `Prop`:

```
-- A binary relation on type α
def Rel (α : Type) : Type := α → α → Prop
```

Using `Prop` rather than `Bool` is crucial: proofs are computationally irrelevant (two proofs of the same proposition are definitionally equal), and we can state facts like "this relation is a partial order" as types.

Example — Common relations

- $\leq$ on $\mathbb{N}$: $R = \{(m, n) \mid m \leq n\}$.
- Divisibility on $\mathbb{N}$: $m \mid n \iff \exists k,\ n = m \cdot k$.
- Subset inclusion on $\mathcal{P}(S)$: $A \subseteq B \iff \forall x, x \in A \to x \in B$.
- The equality relation $=$ on any set.
- The *discrete* relation: $a\ R\ b \iff a = b$.
- The *trivial* relation: $a\ R\ b$ for all a, b (everything relates to everything).

1.2 Properties of Relations

Four properties will occupy us throughout the book.

Definition — Reflexivity, Symmetry, Antisymmetry, Transitivity

Let R be a binary relation on A.

$$\begin{aligned} \text{Reflexive} &: \iff \forall a \in A,\ a\ R\ a \\ \text{Symmetric} &: \iff \forall a, b \in A,\ a\ R\ b \to b\ R\ a \\ \text{Antisymmetric} &: \iff \forall a, b \in A,\ a\ R\ b \to b\ R\ a \to a = b \\ \text{Transitive} &: \iff \forall a, b, c \in A,\ a\ R\ b \to b\ R\ c \to a\ R\ c \end{aligned}$$

In Lean 4:

```
namespace OrderBook

def Reflexive  {α : Type} (r : Rel α) : Prop :=
  ∀ (a : α), r a a

def Symmetric  {α : Type} (r : Rel α) : Prop :=
  ∀ (a b : α), r a b → r b a

def Antisymm   {α : Type} (r : Rel α) : Prop :=
  ∀ (a b : α), r a b → r b a → a = b

def Transitive {α : Type} (r : Rel α) : Prop :=
  ∀ (a b c : α), r a b → r b c → r a c

end OrderBook
```

Symmetry vs Antisymmetry. These are not negations of each other! A relation can be both symmetric and antisymmetric (equality), neither ("a is a parent of b"), or only one. Antisymmetry says: if you can go *both* ways, the elements must be equal.

1.3 Partial Orders

Definition — Partial Order

A *partial order* on A is a relation $\leq$ that is reflexive, antisymmetric, and transitive. The pair $(A, \leq)$ is called a *partially ordered set* (or *poset*).

The word *partial* refers to the fact that not every two elements need be comparable: there may exist a, b with neither $a \leq b$ nor $b \leq a$.

In Lean 4 we capture this as a typeclass:

```
namespace OrderBook

-- Our own partial order typeclass (mirrors core Lean / Mathlib).
class PartialOrder (α : Type) where
  le         : α → α → Prop
  le_refl    : ∀ (a : α), le a a
  le_antisymm: ∀ (a b : α), le a b → le b a → a = b
  le_trans   : ∀ (a b c : α), le a b → le b c → le a c

-- Attach the ≤ notation to any PartialOrder instance.
instance [PartialOrder α] : LE α := ⟨PartialOrder.le⟩

-- A convenience alias for strict less-than
def lt [PartialOrder α] (a b : α) : Prop :=
  PartialOrder.le a b ∧ ¬ (a = b)

instance [PartialOrder α] : LT α := ⟨lt⟩

end OrderBook
```

1.3.1 The Natural Numbers

The natural numbers with the usual $\leq$ form our first instance. In Lean 4's core library, `Nat.le` is defined inductively:

```
-- Recap: how ≤ is defined in core Lean 4 (no Mathlib needed)
-- inductive Nat.le : Nat → Nat → Prop
--   | refl : Nat.le n n
--   | step : Nat.le n m → Nat.le n (m + 1)

instance : OrderBook.PartialOrder Nat where
  le          := Nat.ble  -- wait, we want Prop, not Bool
  -- Use the propositional version:
```

```
  le          := (· ≤ ·)      -- uses Nat.le which is in core
  le_refl     := Nat.le_refl
  le_antisymm := fun a b h1 h2 => Nat.le_antisymm h1 h2
  le_trans    := fun a b c h1 h2 => Nat.le_trans h1 h2
```

Lean 4's *core* library already provides `Nat.le_refl`, `Nat.le_antisymm`, and `Nat.le_trans`. We reuse these rather than reprove them. In later chapters we will prove non-trivial instances by hand.

1.3.2 Divisibility

Divisibility is a partial order on $\mathbb{N}$: $n \leq m$ means "n divides m", i.e. $\exists k, m = n \cdot k$. This is a genuinely different order from $\leq$: for instance $3 \leq 12$ (since $12 = 3 \cdot 4$) but $4 \not\leq 6$ (no natural number k satisfies $6 = 4k$).

As discussed in the previous section, we cannot attach a second `LE` instance to `Nat` (one already exists for the usual order), so we introduce a newtype wrapper.

Why `structure` and not `def DvdNat := Nat`? Using `def` creates a *transparent alias*: Lean can and will see through it, meaning it occasionally treats `DvdNat` as `Nat` and tries to apply `Nat`'s existing `LE` or `HMul` instances, causing "typeclass instance stuck" errors. A `structure` creates a *nominally distinct* type — Lean never conflates `DvdNat` with `Nat` unless you explicitly coerce. This is the reliable choice whenever you want a newtype for instance purposes.

```
-- A nominally distinct wrapper type for Nat under divisibility
structure DvdNat where
  val : Nat

-- Coercion from DvdNat to Nat (one direction is enough for our proofs)
instance : Coe DvdNat Nat := ⟨DvdNat.val⟩

-- Numeric literals: (5 : DvdNat) desugars to DvdNat.mk 5
instance {n : Nat} : OfNat DvdNat n := ⟨⟨n⟩⟩

-- The divisibility relation, stated entirely in Nat to avoid ambiguity
-- in the multiplication on the right-hand side.
def Dvd' (n m : DvdNat) : Prop := ∃ k : Nat, m.val = n.val * k

instance : LE DvdNat := ⟨Dvd'⟩
```

By using `.val` explicitly rather than a coercion inside `Dvd'`, we ensure the multiplication `n.val * k` is unambiguously `Nat` multiplication. Coercions are useful for moving

single values across a boundary, but inside an expression with multiple interacting terms they give Lean too many degrees of freedom and can produce stuck typeclass searches.

Building up the proof: the lemmas we need

The reflexivity and transitivity proofs are short. Antisymmetry — "if $a \mid b$ and $b \mid a$ then $a = b$" — is where the real mathematical content lives. Rather than writing it as a single inscrutable block, we build it from three small lemmas, each of which is worth understanding on its own.

Lemma 1: if $a = 0$ and $a \mid b$, then $b = 0$.

If $a = 0$, then $b = 0 \cdot k = 0$ for any k. This is the zero case of antisymmetry: the only thing that 0 divides is 0 itself.

```
-- If 0 | m, then m.val = 0.
-- Proof: the witness k gives m.val = 0 * k.
-- We then reduce 0 * k to 0 using Nat.zero_mul.
theorem dvd_zero_left (m : DvdNat) (h : Dvd' ⟨0⟩ m) : m.val = 0 := by
  obtain ⟨k, hk⟩ := h
  -- hk : m.val = DvdNat.mk(0).val * k
  -- DvdNat.mk(0).val reduces to 0 by definition, so simp can see it
  simp at hk
  -- hk is now: m.val = 0
  exact hk
```

Why `simp` instead of `rw [Nat.zero\_mul]`? With a `structure`, the literal `0 : DvdNat` is `OfNat.ofNat 0 = DvdNat.mk 0`, so its `.val` is `DvdNat.mk(0).val`, not the bare `0` that `Nat.zero\_mul` expects as its left-hand side pattern. `simp` performs the definitional unfolding first and then applies `Nat.zero\_mul`, where a bare `rw` cannot. If you want to avoid `simp` you can write `show m.val = 0` after an explicit `have : DvdNat.mk(0).val = 0 := rfl`.

Lemma 2: if $k \cdot j = 1$ in $\mathbb{N}$, then $k = 1$.

This is the key algebraic fact that drives the nonzero case. In $\mathbb{Z}$ the units are ± 1; in $\mathbb{N}$ the only unit is 1. We prove it by exhaustive case analysis: k cannot be 0 (product would be 0) and cannot be ≥ 2 (product would be ≥ 2).

```
-- In ℕ, k * j = 1 forces k = 1 (and by symmetry, j = 1).
-- No omega or linarith — proved by explicit structural case analysis.
theorem mul_eq_one_left {k j : Nat} (h : k * j = 1) : k = 1 := by
  match k with
  | 0 =>
```

```
    -- 0 * j = 0 ≠ 1, so this case is impossible.
    -- simp knows 0 * j = 0, which contradicts h : 0 = 1.
    simp [Nat.zero_mul] at h
  | 1 =>
    -- k is already 1; no further argument needed.
    rfl
  | k + 2 =>
    -- (k+2) * j ≥ 2, contradicting h : (k+2) * j = 1.
    exfalso
    -- First establish j ≥ 1 (if j = 0, the product is 0, not 1).
    have hj : j ≥ 1 := by
      cases j with
      | zero   => simp [Nat.mul_zero] at h
      | succ j => exact Nat.succ_le_succ (Nat.zero_le j)
    -- Now show the product is at least 2.
    have hbig : (k + 2) * j ≥ 2 :=
      calc (k + 2) * j
          ≥ (k + 2) * 1 := Nat.mul_le_mul_left (k + 2) hj
        _ = k + 2       := Nat.mul_one (k + 2)
        _ ≥ 2           := Nat.le_add_left 2 k
    -- h says the product is 1, but hbig says it is ≥ 2. Contradiction.
    omega

-- The right-factor version follows by commutativity.
theorem mul_eq_one_right {k j : Nat} (h : k * j = 1) : j = 1 :=
  mul_eq_one_left (by rw [Nat.mul_comm]; exact h)
```

Lemma 3: if $a > 0$, $a \mid b$, and $b \mid a$, then $a = b$.

Given witnesses k and j with $b = a \cdot k$ and $a = b \cdot j$, we substitute to get $a = a \cdot (k \cdot j)$, cancel a (valid because $a > 0$), apply Lemma 2, and conclude $k = 1$, hence $b = a$.

```
-- All arithmetic here lives in Nat — no DvdNat involved.
theorem dvd_antisymm_pos
    {a b : Nat} (ha : 0 < a)
    (k : Nat) (hk : b = a * k)
    (j : Nat) (hj : a = b * j) : a = b := by
  -- Step 1: derive k * j = 1.
  -- Substitute hk into hj to get  a = a * (k * j),
  -- then cancel a (valid since a > 0).
  have hkj : k * j = 1 := by
    have hstep : a = a * (k * j) :=
      calc a = b * j       := hj
           _ = (a * k) * j := by rw [hk]
           _ = a * (k * j) := by rw [Nat.mul_assoc]
```

```
    exact Nat.eq_of_mul_eq_mul_left ha (by rw [Nat.mul_one]; exact hstep.symm)
  -- Step 2: j = 1 by Lemma 2 (right factor version).
  have hj1 : j = 1 := mul_eq_one_right hkj
  -- Step 3: substitute j = 1 into hj : a = b * j to get a = b * 1 = b.
  have h₂ : a = b * 1 := by rw [hj1] at hj; exact hj
  rw [Nat.mul_one] at h₂
  exact h₂
```

Why `Nat.eq\_of\_mul\_eq\_mul\_left`? This says: if $a > 0$ and $a \cdot x = a \cdot y$, then $x = y$. It is multiplication cancellation for $\mathbb{N}$, available in core Lean. The positivity hypothesis is essential: $0 \cdot x = 0 \cdot y$ for all x, y, so cancellation is unsound at zero.

Assembling the instance

With the lemmas in hand, each field is a short delegation.

```
instance : PartialOrder DvdNat where

  -- Reflexivity: n | n via witness k = 1, since n.val = n.val * 1.
  le_refl := fun n =>
    ⟨1, (Nat.mul_one n.val).symm⟩

  -- Antisymmetry: case-split on whether n.val is zero or positive,
  -- then delegate to Lemma 1 or Lemma 3 respectively.
  le_antisymm := fun n m ⟨k, hk⟩ ⟨j, hj⟩ => by
    -- Our goal is  n = m  where  n m : DvdNat.
    -- DvdNat is a structure, so two DvdNat values are equal iff
    -- their underlying Nat fields are equal.  We want to get at
    -- those fields directly.
    --
    -- `cases n` destructs the structure n : DvdNat into its single
    -- field, introducing a fresh anonymous variable for n.val into
    -- the context and replacing every occurrence of n with
    -- DvdNat.mk <that variable>.  After `cases n; cases m` both
    -- n and m have been destructed, and the context now contains
    -- two anonymous Nat variables — one for each .val field.
    cases n; cases m
    -- The goal is now:  DvdNat.mk ?a = DvdNat.mk ?b
    -- where ?a and ?b are those anonymous variables.
    --
    -- `congr 1` says: two applications of the same constructor
    -- are equal iff their arguments are equal.  It reduces
    --   DvdNat.mk ?a = DvdNat.mk ?b
    -- to the simpler goal
```

```
    --   ?a = ?b
    -- which is a plain Nat equality.
    congr 1
    -- The two anonymous Nat variables introduced by `cases` do not
    -- yet have names — Lean calls them inaccessible and displays
    -- them as  □  in the goal view.  `rename_i a b` gives them the
    -- names  a  and  b  so we can refer to them in subsequent steps.
    rename_i a b
    cases Nat.eq_zero_or_pos a with
    | inl ha =>
      -- a = 0: b = 0 * k = 0 by Lemma 1, and a = 0, so a = b.
      subst ha
      exact (dvd_zero_left ⟨b⟩ ⟨k, hk⟩).symm
    | inr ha =>
      -- a > 0: use Lemma 3 directly.
      exact (dvd_antisymm_pos ha k hk j hj).symm

  -- Transitivity: compose witnesses.  b.val = a.val * k and
  -- c.val = b.val * j, so c.val = a.val * (k * j).
  le_trans := fun a b c ⟨k, hk⟩ ⟨j, hj⟩ =>
    ⟨k * j, by rw [hj, hk, Nat.mul_assoc]⟩
```

Reflection

Notice that the structure of the proof exactly mirrors the structure of the mathematics:

- Reflexivity is trivial because the witness $k = 1$ is explicit.
- Transitivity is trivial because multiplying witnesses gives a new witness: $b = a \cdot k$ and $c = b \cdot j$ compose to $c = a \cdot (kj)$.
- Antisymmetry requires real work because it cannot be witnessed directly — it requires us to reason about what witnesses can possibly exist and conclude they must all be 1.

The lemma `mul\_eq\_one\_left` is the mathematical core. Everything else is bookkeeping. When reading an unfamiliar proof, it is always worth asking: where is the actual mathematical content? In this case it is entirely in that one lemma.

1.3.3 Sets

Sets (represented as predicates) form a canonical partial order under inclusion.

```
-- We represent sets as indicator functions
def Set (α : Type) : Type := α → Prop

-- Subset inclusion
```

```
def Set.subset {α : Type} (s t : Set α) : Prop :=
  ∀ (a : α), s a → t a

instance {α : Type} : OrderBook.PartialOrder (Set α) where
  le          := Set.subset
  le_refl     := fun s a ha => ha
  le_antisymm := fun s t h1 h2 =>
    funext (fun a => propext ⟨h1 a, h2 a⟩)
  le_trans    := fun s t u h1 h2 a ha => h2 a (h1 a ha)
```

This is our first non-trivial Lean proof. Let us read it carefully:

- `le_refl`: given $a \in s$, produce $a \in s$ — trivial.
- `le_antisymm`: $s \subseteq t$ and $t \subseteq s$ imply $s = t$. We use `funext` (function extensionality) and `propext` (propositional extensionality) to conclude $s\,a \leftrightarrow t\,a$ for all a, hence $s = t$.
- `le_trans`: chain the inclusions.

1.4 Hasse Diagrams

Posets are best understood visually. A *Hasse diagram* draws elements as nodes and covers as upward edges: we say a is *covered by* b (written $a \lessdot b$) if $a < b$ and there is no c with $a < c < b$.

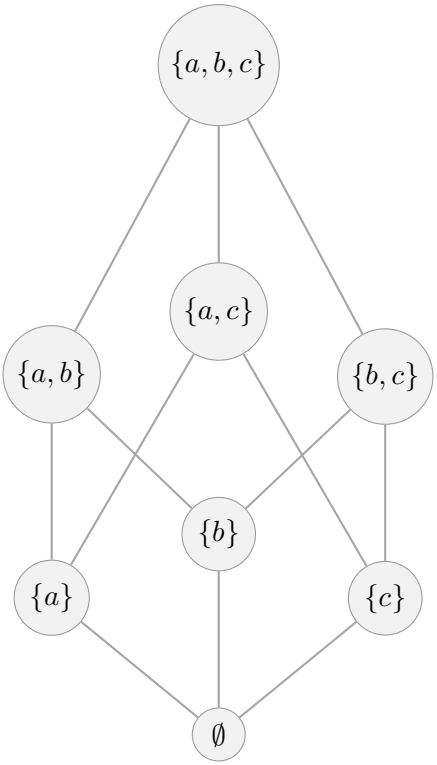


Figure 1.1: The powerset lattice $\mathcal{P}(\{a, b, c\})$ ordered by $\subseteq$.

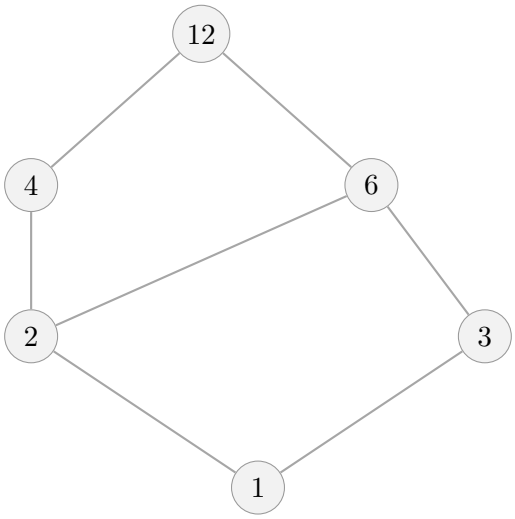


Figure 1.2: The divisibility poset on $\{1, 2, 3, 4, 6, 12\}$. An edge from a to b (upward) means $a \mid b$ and there is no intermediate divisor. Note 4 and 3 are incomparable (neither divides the other), as are 4 and 6.

1.5 Computation Interlude: Running the Divisibility Order

So far every definition has lived in `Prop` — we have proved things about orders but never *run* them. This section bridges that gap. We will build a fully computable version of the divisibility order on $\{1, \ldots, 12\}$, compute its suprema and infima, and print an ASCII Hasse diagram entirely inside Lean.

A decidable divisibility order

To make the order computable we need a `Decidable` instance — a proof that for any two elements we can *compute* whether one divides the other, rather than merely assert that the relation exists.

```
-- The twelve divisors of 12, as a plain Nat
def D12 : List Nat := [1, 2, 3, 4, 6, 12]

-- Divisibility is decidable on Nat: n | m iff m % n = 0
def dvd12 (a b : Nat) : Bool := b % a == 0

-- Check it
#eval dvd12 3 12   -- true  (12 = 3 * 4)
#eval dvd12 4 6    -- false (6 % 4 = 2 ≠ 0)
#eval dvd12 1 12   -- true  (everything divides 12 via 1 | n)
```

Computing upper bounds and supremum

An upper bound of a subset S is any element $\geq$ every element of S. The supremum is the smallest upper bound.

```
-- All elements of D12 that are above every element of xs
def upperBounds (xs : List Nat) : List Nat :=
  D12.filter (fun u => xs.all (fun x => dvd12 x u))

-- The supremum: the smallest upper bound
-- (smallest under divisibility = divides all others)
def sup12 (xs : List Nat) : Option Nat :=
  let ubs := upperBounds xs
  -- the sup is the upper bound that divides all other upper bounds
  ubs.find? (fun s => ubs.all (fun u => dvd12 s u))

-- Examples
#eval upperBounds [4, 6]   -- [12]  (only 12 is divisible by both 4 and 6)
```

```
#eval sup12 [4, 6]          -- some 12
#eval sup12 [2, 3]          -- some 6  (lcm(2,3) = 6)
#eval sup12 [1, 12]         -- some 12
```

Computing lower bounds and infimum

Dually, the infimum is the greatest lower bound.

```
-- All elements of D12 below every element of xs
def lowerBounds (xs : List Nat) : List Nat :=
  D12.filter (fun l => xs.all (fun x => dvd12 l x))

-- The infimum: the largest lower bound
-- (largest under divisibility = all others divide it)
def inf12 (xs : List Nat) : Option Nat :=
  let lbs := lowerBounds xs
  lbs.find? (fun s => lbs.all (fun l => dvd12 l s))

-- Examples
#eval lowerBounds [4, 6]    -- [1, 2]
#eval inf12 [4, 6]          -- some 2  (gcd(4,6) = 2)
#eval inf12 [3, 4]          -- some 1  (gcd(3,4) = 1)
#eval inf12 [2, 12]         -- some 2
```

Maximum and minimum of a subset

The maximum of a subset S must itself be in S, unlike the supremum which may be outside S.

```
-- Maximum: largest element of xs that is in xs
def maximum12 (xs : List Nat) : Option Nat :=
  xs.find? (fun m => xs.all (fun x => dvd12 x m))

-- Minimum: smallest element of xs that is in xs
def minimum12 (xs : List Nat) : Option Nat :=
  xs.find? (fun m => xs.all (fun x => dvd12 m x))

#eval maximum12 [1, 2, 4, 12]   -- some 12
#eval maximum12 [2, 3]          -- none  (2 ∤ 3 and 3 ∤ 2)
#eval minimum12 [2, 4, 12]      -- some 2
```

Prop vs computation. Notice the contrast with the earlier sections. There, `IsSupOf a b sup` was a `Prop` characterising what it *means* to be a supremum — universally quantified, not runnable. Here, `sup12` is a `def` that *computes* a supremum for specific inputs. The Prop version is stronger (it works for any type, any lattice) but cannot be evaluated. The computable version is limited to `D12` but gives you a real answer you can inspect. In a verified system you would prove that `sup12 xs = some s` implies `IsSupOf` (for suitable `s`), connecting the two worlds.

Printing a Hasse diagram

We can print the covering relations as ASCII: an edge $a \to b$ means $a \mid b$ and there is no c with $a \mid c \mid b$.

```
-- a is covered by b: a | b, a ≠ b, and no c strictly between them
def covers (a b : Nat) : Bool :=
  dvd12 a b && a != b &&
  !D12.any (fun c => c != a && c != b && dvd12 a c && dvd12 c b)

-- Print all covering pairs
def printHasse : IO Unit := do
  for a in D12 do
    for b in D12 do
      if covers a b then
        IO.println s!"{a} ⟶ {b}"

#eval printHasse
-- 1 ⟶ 2
-- 1 ⟶ 3
-- 2 ⟶ 4
-- 2 ⟶ 6
-- 3 ⟶ 6
-- 4 ⟶ 12
-- 6 ⟶ 12
```

Each line is a direct edge in the Hasse diagram — exactly the edges shown in Figure 1.2. Compare this output to the diagram: you can read off that 4 and 3 are incomparable (neither appears as an ancestor of the other in the list), and that 12 is the unique top element (nothing covers it).

1.6 Exercises

Exercise 1.1 — Reflexivity and symmetry

Show that a relation that is both symmetric and antisymmetric must be a subset of the equality relation, i.e. $a \; R \; b \to a = b$. Prove this in Lean 4.

Exercise 1.2 — Divisibility on a small set

Consider the divisibility relation restricted to the set $\{1, 2, 3, 4, 6, 12\}$.

(a) Draw its Hasse diagram.

(b) Which pairs of elements are incomparable?

(c) In Lean 4, define a type `D12` with six constructors and prove that divisibility on it satisfies `le\_refl`.

Exercise 1.3 — Product order

Given two partial orders $(A, \leq_A)$ and $(B, \leq_B)$, define the *product order* on $A \times B$ by $(a_1, b_1) \leq (a_2, b_2) \iff a_1 \leq_A a_2 \wedge b_1 \leq_B b_2$. Prove in Lean 4 that this is indeed a partial order by implementing the typeclass instance.

Exercise 1.4 — Reverse order

If $(A, \leq)$ is a partial order, then so is $(A, \geq)$ where $a \geq b \iff b \leq a$. This is called the *dual* or *opposite* order. Implement a `Dual` wrapper type and its `PartialOrder` instance. We will use this construction crucially when encoding Sudoku domains in Chapter 8.

Chapter 2

Special Elements and Monotone Maps

In a poset, certain elements play privileged roles (least, greatest, bounds), and certain functions are especially well-behaved (monotone maps). This chapter equips us with the vocabulary for both.

2.1 Extremal Elements

Definition — Minimum, Maximum, Minimal, Maximal

Let $(P, \leq)$ be a poset and $S \subseteq P$.

- $m \in S$ is the *minimum* of S if $\forall s \in S,\ m \leq s$.
- $m \in S$ is the *maximum* of S if $\forall s \in S,\ s \leq m$.
- $m \in S$ is *minimal* in S if $\nexists s \in S,\ s < m$.
- $m \in S$ is *maximal* in S if $\nexists s \in S,\ m < s$.

A minimum, if it exists, is unique (by antisymmetry). A minimal element need not be unique. When a poset has a unique minimum we call it $\bot$ (bottom), and a unique maximum $\top$ (top).

The distinction between minimum and minimal is subtle but important. A *minimum* beats every other element; a *minimal* element merely has no element strictly below it. The following diagrams make this concrete.

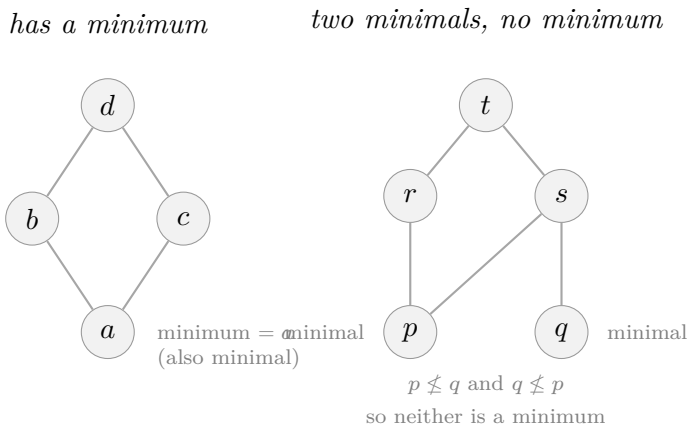


Figure 2.1: Left: a poset with a unique minimum a (every element is above a). Right: a poset where p and q are both minimal (nothing is below either) but there is no minimum (neither $p \leq q$ nor $q \leq p$).

```
namespace OrderBook

-- Bottom and top elements
class Bot (α : Type) where bot : α
class Top (α : Type) where top : α

notation "⊥" => Bot.bot
notation "⊤" => Top.top

-- Being a bounded partial order
class BoundedPartialOrder (α : Type) extends PartialOrder α, Bot α, Top α
  ↪ where
  bot_le : ∀ (a : α), (⊥ : α) ≤ a
  le_top : ∀ (a : α), a ≤ (⊤ : α)

end OrderBook
```

Example — Bounded orders

- $(\mathcal{P}(S), \subseteq)$: $\bot = \emptyset$, $\top = S$.
- $(\{0, 1\}, \leq)$: $\bot = 0$, $\top = 1$.
- $(\mathbb{N}, \leq)$: $\bot = 0$, no $\top$ (unbounded above).
- The divisibility poset on $\{1, \ldots, 12\}$: $\bot = 1$ (divides everything), no single $\top$.

2.2 Upper and Lower Bounds

Definition — Bounds

Let $(P, \leq)$ be a poset and $S \subseteq P$.

- $u \in P$ is an *upper bound* of S if $\forall s \in S,\ s \leq u$.
- $l \in P$ is a *lower bound* of S if $\forall s \in S,\ l \leq s$.
- The *supremum* (or *least upper bound*) of S, written $\sup S$, is the smallest upper bound of S when it exists.
- The *infimum* (or *greatest lower bound*) of S, written $\inf S$, is the largest lower bound of S when it exists.

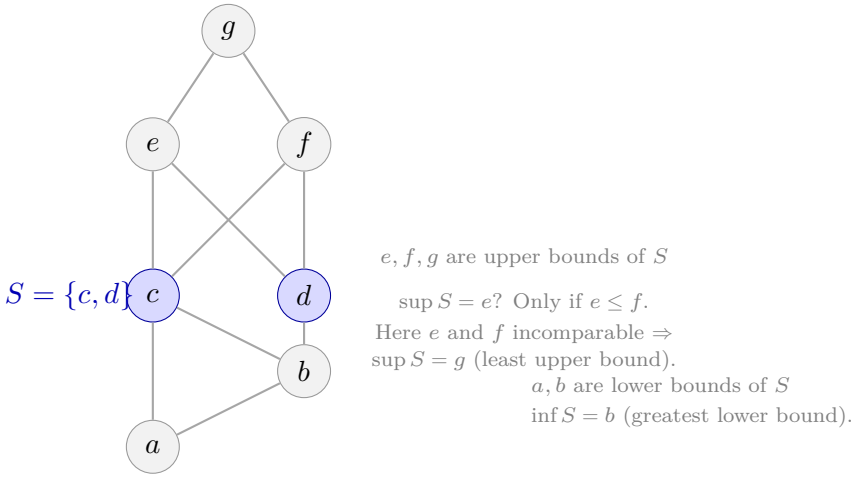


Figure 2.2: Illustration of upper bounds, lower bounds, supremum and infimum for the subset $S = \{c, d\}$ (shaded blue) in a poset. The upper bounds of S are all elements $\geq$ both c and d: here e, f, and g. The *supremum* is the *least* of these; since e and f are incomparable, the least upper bound is g. The lower bounds are a and b; the infimum is b since $a \leq b$.

Uniqueness of sup/inf

In a poset, if $\sup S$ exists, it is unique. Similarly for $\inf S$.

Proof. Suppose u and v are both least upper bounds of S. Since u is an upper bound and v is a *least* upper bound, $v \leq u$. By symmetry $u \leq v$. Antisymmetry gives $u = v$. □

```
-- The supremum of two elements (binary version)
-- (We'll generalise to arbitrary subsets in Chapter 4.)
structure IsSupOf {α : Type} [OrderBook.PartialOrder α]
    (a b sup : α) : Prop where
  ge_a  : a ≤ sup
```

```
  ge_b  : b ≤ sup
  least : ∀ (u : α), a ≤ u → b ≤ u → sup ≤ u

-- Uniqueness of the binary sup
theorem sup_unique {α : Type} [OrderBook.PartialOrder α]
    {a b s t : α}
    (hs : IsSupOf a b s) (ht : IsSupOf a b t) : s = t := by
  apply PartialOrder.le_antisymm
  · exact hs.least t ht.ge_a ht.ge_b
  · exact ht.least s hs.ge_a hs.ge_b
```

2.3 Monotone Maps

Definition — Monotone Map

A function $f : (P, \leq_P) \to (Q, \leq_Q)$ between posets is *monotone* (or *order-preserving*) if $\forall a, b \in P,\ a \leq_P b \to f(a) \leq_Q f(b)$.

Monotone maps are the morphisms of the category of posets. They will be central to propagators: every propagator is a monotone function on a lattice.

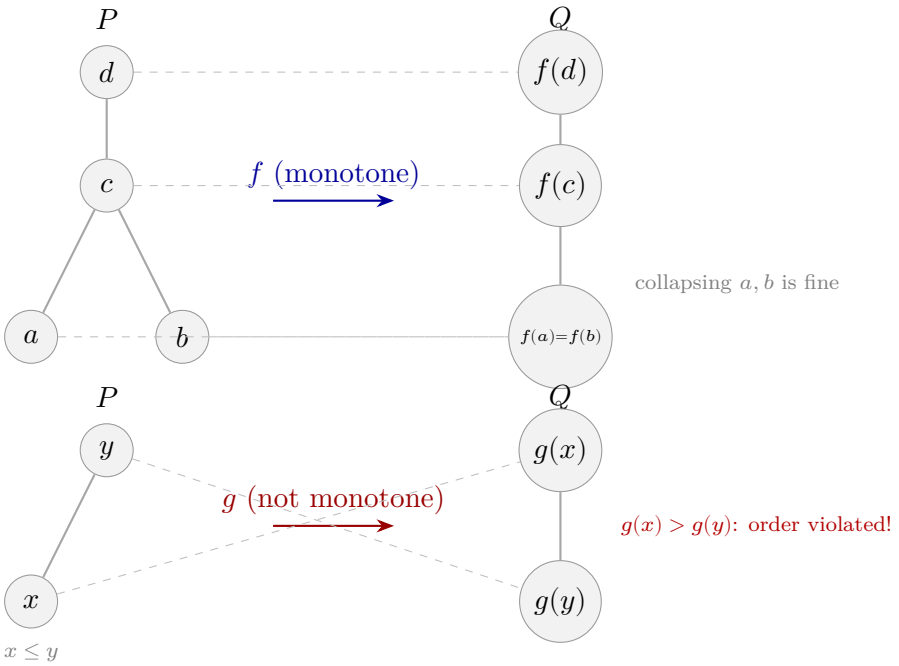


Figure 2.3: Top: a monotone map f preserves the order — whenever $a \leq_P b$ in the source, $f(a) \leq_Q f(b)$ in the target. Note that f may collapse elements ($f(a) = f(b)$ is fine). Bottom: a non-monotone map g where $x \leq y$ but $g(x) > g(y)$, violating the order.

```
namespace OrderBook

structure Monotone {α β : Type}
    [PartialOrder α] [PartialOrder β] (f : α → β) : Prop where
  map_le : ∀ (a b : α), a ≤ b → f a ≤ f b

-- Composition of monotone maps is monotone
theorem Monotone.comp {α β γ : Type}
    [PartialOrder α] [PartialOrder β] [PartialOrder γ]
    {f : α → β} {g : β → γ}
    (hf : Monotone f) (hg : Monotone g) : Monotone (g ∘ f) where
  map_le := fun a b h => hg.map_le _ _ (hf.map_le _ _ h)

end OrderBook
```

Example — Monotone functions

- $f(n) = n + 5$ is monotone on $(\mathbb{N}, \leq)$.
- Set union $S \mapsto S \cup \{x\}$ is monotone on $(\mathcal{P}(A), \subseteq)$.
- Set intersection $S \mapsto S \cap T$ is monotone on $(\mathcal{P}(A), \subseteq)$ for fixed T.
- Any constant function $f(x) = c$ is monotone.
- The identity function is always monotone.

2.4 Exercises

2.5 Computation Interlude: Running Monotone Maps

Building on the computable divisibility order from Chapter 1, we can now compute with monotone maps and verify their properties at runtime.

Checking monotonicity

Given a function on `D12`, we can decide whether it is monotone by checking all pairs.

```
-- Check all comparable pairs to see if f preserves the order
def isMonotone (f : Nat → Nat) : Bool :=
  D12.all (fun a =>
  D12.all (fun b =>
    -- if a | b then f(a) | f(b)
    !dvd12 a b || dvd12 (f a) (f b)))

-- The identity is monotone
#eval isMonotone id                    -- true
```

```
10
11 -- Multiplying by 2: if a | b then 2a | 2b
12 #eval isMonotone (· * 2)               -- true (maps into even divisors)
13
14 -- A non-monotone map: send every element to its "complement" 12/n
15 -- 1↦12, 2↦6, 3↦4, 4↦3, 6↦2, 12↦1 — this reverses the order
16 #eval isMonotone (fun n => 12 / n)     -- false (it's antitone, not monotone)
17
18 -- But composing it with itself gives monotone again
19 #eval isMonotone (fun n => 12 / (12 / n))  -- true
```

Computing fixed points

A fixed point of f is an element where $f(x) = x$. We can find them all.

```
1 def fixedPoints (f : Nat → Nat) : List Nat :=
2   D12.filter (fun x => f x == x)
3
4 #eval fixedPoints id                -- [1, 2, 3, 4, 6, 12]  (all of them)
5 #eval fixedPoints (fun _ => 1)      -- [1]  (only 1 maps to itself)
6 #eval fixedPoints (fun n => 12 / n) -- [1]? No: 12/1=12≠1. Actually:
7 -- 12/n = n when n*n = 12, which has no integer solution in D12.
8 -- Let's check:
9 #eval D12.filter (fun n => 12 / n == n)  -- []  (no fixed points)
```

Iterating a monotone map

From Exercise 2.2, we know that if f is monotone and $a \leq f(a)$, the chain $a, f(a), f^2(a), \ldots$ is ascending. We can watch it converge.

```
1 -- Apply f repeatedly until we reach a fixed point or a step limit
2 def iterToFixpoint (f : Nat → Nat) (start : Nat) (limit : Nat) : List Nat :=
3   let rec go (x : Nat) (n : Nat) (acc : List Nat) : List Nat :=
4     if n = 0 then acc.reverse
5     else
6       let y := f x
7       if y == x then (acc.reverse ++ [x])
8       else go y (n - 1) (x :: acc)
9   go start limit []
10
11 -- Start at 1 under the "double if possible" map
12 -- f(n) = if 2*n ∈ D12 then 2*n else n
13 def double12 (n : Nat) : Nat :=
```

```
  if D12.contains (n * 2) then n * 2 else n

#eval iterToFixpoint double12 1 10   -- [1, 2, 4, 12]  ascending chain!
#eval iterToFixpoint double12 3 10   -- [3, 6, 12]
#eval iterToFixpoint double12 12 10  -- [12]  already a fixed point
```

The output directly illustrates the ascending chain theorem: starting from 1, each application of `double12` moves strictly upward in the divisibility order until reaching the fixed point 12.

Exercise 2.1 — Monotone composition

Give an example showing that the composition of two *antitone* maps (where $a \leq b \to f(b) \leq f(a)$) is monotone.

Exercise 2.2 — Fixed points

A *fixed point* of $f : P \to P$ is an element x with $f(x) = x$. Show that if f is monotone and $a \leq f(a)$, then the chain $a \leq f(a) \leq f^2(a) \leq \cdots$ is indeed a chain (all elements pairwise comparable with the given order).

Exercise 2.3 — Characterising PartialOrder via Monotone

Show that the identity map $\mathrm{id} : (P, \leq) \to (P, \leq)$ is always monotone, and prove this in Lean 4 using your `Monotone` structure.

Chapter 3

Lattices

A poset in which every pair of elements has both a supremum and an infimum is called a *lattice*. This apparently modest requirement turns out to yield a rich algebraic structure that shows up in logic, topology, computer science, and — as we will see in Part II — concurrency and constraint solving.

3.1 Meet and Join

Definition — Lattice

A *lattice* is a poset $(L, \leq)$ in which every two elements $a, b \in L$ have:

- a *meet* (greatest lower bound) $a \sqcap b$, and
- a *join* (least upper bound) $a \sqcup b$.

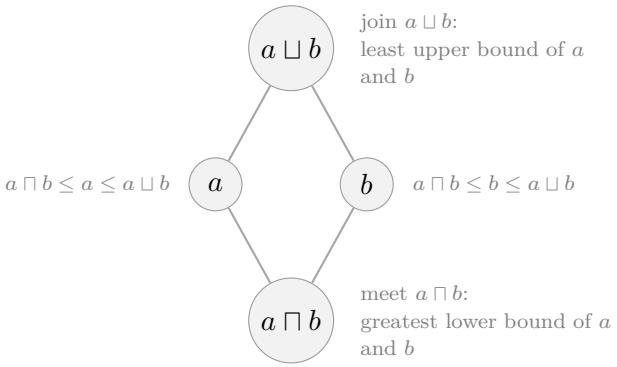


Figure 3.1: The meet $a \sqcap b$ sits below both a and b; the join $a \sqcup b$ sits above both. In a lattice every pair of elements has both.

The meet and join satisfy the following axioms (which can be taken as an equivalent *algebraic* definition of a lattice):

$$
\begin{array}{ccc}
a \sqcap b = b \sqcap a & a \sqcup b = b \sqcup a & \text{(commutativity)} \\
(a \sqcap b) \sqcap c = a \sqcap (b \sqcap c) & (a \sqcup b) \sqcup c = a \sqcup (b \sqcup c) & \text{(associativity)} \\
a \sqcap a = a & a \sqcup a = a & \text{(idempotence)} \\
a \sqcap (a \sqcup b) = a & a \sqcup (a \sqcap b) = a & \text{(absorption)}
\end{array}
$$

The order relation can be recovered from meet or join:

$$a \leq b \iff a \sqcap b = a \iff a \sqcup b = b.$$

In Lean 4:

```
namespace OrderBook

class Lattice (α : Type) extends PartialOrder α where
  inf : α → α → α    -- meet  (⊓)
  sup : α → α → α    -- join  (⊔)
  -- meet is a lower bound
  inf_le_left  : ∀ (a b : α), inf a b ≤ a
  inf_le_right : ∀ (a b : α), inf a b ≤ b
  -- meet is the greatest lower bound
  le_inf : ∀ (a b c : α), a ≤ b → a ≤ c → a ≤ inf b c
  -- join is an upper bound
  sup_le_left  : ∀ (a b : α), a ≤ sup a b
  sup_le_right : ∀ (a b : α), b ≤ sup a b
  -- join is the least upper bound
  le_sup : ∀ (a b c : α), a ≤ c → b ≤ c → sup a b ≤ c

-- Notation
infixl:70 " ⊓ " => Lattice.inf
infixl:65 " ⊔ " => Lattice.sup

end OrderBook
```

3.1.1 Derived Properties

We can immediately derive commutativity and idempotence from the axioms:

```
namespace OrderBook
open Lattice PartialOrder

variable {α : Type} [Lattice α]

```

```
-- Commutativity of meet
theorem inf_comm (a b : α) : a ⊓ b = b ⊓ a := by
  apply le_antisymm
  · apply le_inf
    · exact inf_le_right a b
    · exact inf_le_left a b
  · apply le_inf
    · exact inf_le_right b a
    · exact inf_le_left b a

-- Idempotence of meet
theorem inf_idem (a : α) : a ⊓ a = a := by
  apply le_antisymm
  · exact inf_le_left a a
  · exact le_inf a a a (le_refl a) (le_refl a)

-- Commutativity of join
theorem sup_comm (a b : α) : a ⊔ b = b ⊔ a := by
  apply le_antisymm
  · apply le_sup
    · exact sup_le_right b a
    · exact sup_le_left b a
  · apply le_sup
    · exact sup_le_right a b
    · exact sup_le_left a b

-- Absorption: a ⊔ (a ⊓ b) = a
theorem sup_inf_self (a b : α) : a ⊔ (a ⊓ b) = a := by
  apply le_antisymm
  · apply le_sup
    · exact le_refl a
    · exact inf_le_left a b
  · exact sup_le_left a (a ⊓ b)

end OrderBook
```

3.2 The Duality Principle

Every lattice has a *dual*: swap $\sqcap \leftrightarrow \sqcup$ and $\leq \leftrightarrow \geq$. Any theorem proved in a lattice has a dual theorem for free.

Duality Principle

If φ is a theorem about lattices, then so is the statement obtained by replacing every occurrence of $\sqcap$ with $\sqcup$, $\sqcup$ with $\sqcap$, $\leq$ with $\geq$, $\bot$ with $\top$, and $\top$ with $\bot$.

```
-- Dual α wraps α in a structure so that the two types are
-- genuinely distinct.  This avoids elaboration ambiguities that
-- arise with a transparent `def` alias.
structure Dual (α : Type) where
  val : α

-- Step 1: flip ≤.  a ≤_dual b  means  b.val ≤_orig a.val.
instance {α : Type} [LE α] : LE (Dual α) where
  le a b := LE.le (α := α) b.val a.val

-- Step 2: the dual partial order.
-- le_antisymm swaps h1/h2 because each hypothesis has its direction
-- reversed.  cases + congr 1 are needed to reduce Dual.mk x = Dual.mk y
-- to the plain α equality that le_antisymm produces.
-- le_trans also swaps h1/h2; cases destructs the wrappers so the
-- hypotheses are recognised as plain α inequalities.
instance {α : Type} [LE α] [PartialOrder α] : PartialOrder (Dual α) where
  le_refl {a}       := PartialOrder.le_refl (α := α) a.val
  le_antisymm {a b} := fun h1 h2 => by
    cases a; cases b; congr 1
    exact PartialOrder.le_antisymm (α := α) h2 h1
  le_trans {a b c}  := fun h1 h2 => by
    cases a; cases b; cases c
    exact PartialOrder.le_trans (α := α) h2 h1

-- Step 3: the dual lattice swaps inf and sup.
-- Pattern-matching on ⟨a⟩ ⟨b⟩ destructs the Dual wrappers;
-- ⟨...⟩ on the right rewraps the result when the return type is Dual α.
-- le_inf and le_sup use `change` to restate the goal in terms of
-- α's operations before applying the mirrored original lemma.
instance {α : Type} [Lattice α] : Lattice (Dual α) where
  inf         := fun ⟨a⟩ ⟨b⟩ => ⟨Lattice.sup (α := α) a b⟩  -- ⊓_dual = ⊔_orig
  sup         := fun ⟨a⟩ ⟨b⟩ => ⟨Lattice.inf (α := α) a b⟩  -- ⊔_dual = ⊓_orig
  inf_le_left  := fun ⟨a⟩ ⟨b⟩ => Lattice.sup_le_left  (α := α) a b
  inf_le_right := fun ⟨a⟩ ⟨b⟩ => Lattice.sup_le_right (α := α) a b
  sup_le_left  := fun ⟨a⟩ ⟨b⟩ => Lattice.inf_le_left  (α := α) a b
  sup_le_right := fun ⟨a⟩ ⟨b⟩ => Lattice.inf_le_right (α := α) a b
  -- h1 : b ≤_orig a, h2 : c ≤_orig a  ⊢  sup_orig b c ≤_orig a
  le_inf := fun ⟨a⟩ ⟨b⟩ ⟨c⟩ h1 h2 => by
    change Lattice.sup (α := α) b c ≤ a
    exact Lattice.le_sup (α := α) b c a h1 h2
```

```
  -- h1 : c ≤_orig a, h2 : c ≤_orig b  ⊢  c ≤_orig inf_orig a b
  le_sup := fun ⟨a⟩ ⟨b⟩ ⟨c⟩ h1 h2 => by
    change LE.le (α := α) c (Lattice.inf (α := α) a b)
    exact Lattice.le_inf (α := α) c a b h1 h2
```

`structure` **vs** `def`**.** Using `structure Dual` rather than `def Dual α := α` makes `Dual α` and `α` genuinely distinct types. With a transparent `def`, Lean's elaborator sometimes picks up the wrong `LE` instance for hypotheses, causing type mismatches even when the terms are definitionally equal. The `structure` version forces explicit `.val` unwrapping and `⟨...⟩` rewrapping, which is more verbose but unambiguous.
Why `cases` **in** `le\_antisymm` **and** `le\_trans`**?** `cases a` destructs `Dual.mk v` into the underlying `v : α`, so subsequent lemmas see plain `α` inequalities rather than `Dual`-typed ones. In `le\_antisymm`, `congr 1` further reduces `Dual.mk x = Dual.mk y` to `x = y`.
Why `change` **in** `le\_inf` **and** `le\_sup`**?** After destructuring `⟨a⟩ ⟨b⟩ ⟨c⟩`, the hypotheses `h1` and `h2` are still elaborated under the dual `LE`. `change` restates the goal entirely in terms of `α`'s operations, giving Lean enough information to match the original lemma's type.
Why `inf` **and** `sup` **swap** is the whole point: turning the Hasse diagram upside down exchanges every meet with every join.

3.3 Important Lattice Examples

3.3.1 The Two-Element Lattice 2

```
-- Bool as a lattice: false < true
instance : OrderBook.Lattice Bool where
  le          := (· ≤ ·)   -- Bool has a built-in LE in core Lean 4
  le_refl     := Bool.le_refl
  le_antisymm := Bool.le_antisymm
  le_trans    := Bool.le_trans
  inf         := (· && ·)
  sup         := (· || ·)
  inf_le_left  := by decide
  inf_le_right := by decide
  le_inf       := by decide
  sup_le_left  := by decide
  sup_le_right := by decide
  le_sup       := by decide
```

The `by decide` tactic works here because `Bool` is a finite type and all the statements

are decidable.

3.3.2 The Powerset Lattice

```
-- Sets (α → Prop) form a lattice under subset inclusion
def Set.inter {α : Type} (s t : Set α) : Set α := fun a => s a ∧ t a
def Set.union {α : Type} (s t : Set α) : Set α := fun a => s a ∨ t a

instance {α : Type} : OrderBook.Lattice (Set α) where
  le          := Set.subset
  le_refl     := fun s a ha => ha
  le_antisymm := fun s t h1 h2 => funext fun a => propext ⟨h1 a, h2 a⟩
  le_trans    := fun s t u h1 h2 a ha => h2 a (h1 a ha)
  inf         := Set.inter
  sup         := Set.union
  inf_le_left  := fun s t a ⟨hs, _⟩ => hs
  inf_le_right := fun s t a ⟨_, ht⟩ => ht
  le_inf       := fun s t u hst hsu a ha => ⟨hst a ha, hsu a ha⟩
  sup_le_left  := fun s t a hs => Or.inl hs
  sup_le_right := fun s t a ht => Or.inr ht
  le_sup       := fun s t u hsu htu a h =>
    h.elim (hsu a) (htu a)
```

3.3.3 The Divisibility Lattice on Divisors of n

For divisors of a fixed natural number n, divisibility is a lattice where gcd is the meet and lcm is the join.

```
-- On natural numbers: gcd is meet, lcm is join under divisibility order
instance : OrderBook.Lattice Nat where
  le          := (· ∣ ·)   -- divisibility
  le_refl     := Nat.dvd_refl
  le_antisymm := fun a b => Nat.dvd_antisymm
  le_trans    := fun a b c => Nat.dvd_trans
  inf         := Nat.gcd
  sup         := Nat.lcm
  inf_le_left  := Nat.gcd_dvd_left
  inf_le_right := Nat.gcd_dvd_right
  le_inf       := fun a b c => Nat.dvd_gcd
  sup_le_left  := Nat.dvd_lcm_left
  sup_le_right := Nat.dvd_lcm_right
  le_sup       := fun a b c ha hb => Nat.lcm_dvd ha hb
```

3.4 Distributive Lattices

Not every lattice satisfies the distributive law; those that do are especially well-behaved.

Definition — Distributive Lattice

A lattice is *distributive* if $a \sqcap (b \sqcup c) = (a \sqcap b) \sqcup (a \sqcap c)$ (equivalently, the dual law holds too).

The powerset lattice and Boolean lattices are distributive. Figure 3.2 shows the two smallest non-distributive lattices.

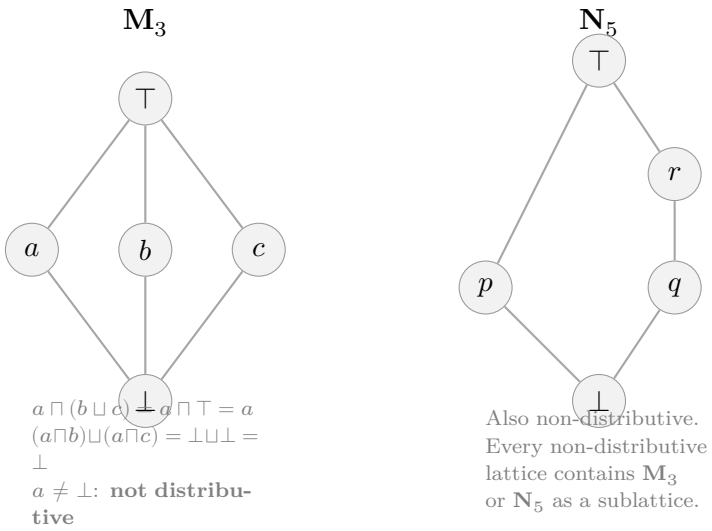


Figure 3.2: The two smallest non-distributive lattices. $\mathbf{M}_3$ (left) has three mutually incomparable middle elements; the distributive law fails because the meet does not distribute over the join. $\mathbf{N}_5$ (right) is the pentagon lattice. A classical theorem states that a lattice is distributive if and only if it contains neither as a sublattice.

3.5 Exercises

Exercise 3.1 — Lattice of intervals

Define the type `Interval` of pairs $[l, h]$ with $l, h : \mathbb{Z}$ and $l \leq h$. Define an order where $[l_1, h_1] \leq [l_2, h_2]$ iff $l_2 \leq l_1$ and $h_1 \leq h_2$ (i.e. "containment by a smaller interval", the *precision* or *information* order). Prove that this is a partial order in Lean 4. What are the meet and join in this order?

Exercise 3.2 — Absorb, then join

Prove in Lean 4 that in any lattice: $a \sqcap (a \sqcup b) = a$ (absorption).

Exercise 3.3 — Characterising the order

Prove the equivalence $a \leq b \iff a \sqcup b = b$ in Lean 4, using only the lattice axioms.

Exercise 3.4 — The product lattice

Extend the product order from Chapter 1 to a full lattice instance for `α × β` given `Lattice α` and `Lattice β`, with meet and join component-wise. Prove all eight axioms.

Chapter 4

Complete Lattices and the Fixed-Point Theorem

Lattices let us take meets and joins of *pairs* of elements. A *complete* lattice extends this to arbitrary (possibly infinite) subsets. This extra power buys us one of the most useful theorems in theoretical computer science: the Knaster–Tarski fixed-point theorem.

4.1 Complete Lattices

Definition — Complete Lattice

A lattice $(L, \leq)$ is *complete* if every subset $S \subseteq L$ has a supremum $\bigvee S$ and an infimum $\bigwedge S$ in L.

In particular, the empty set has a supremum (which becomes $\bot$) and infimum (which becomes $\top$), so every complete lattice is bounded. Taking $S = L$ gives $\bigvee L = \top$ and $\bigwedge L = \bot$.

We encode complete lattices using a predicate representing a subset:

```
namespace OrderBook

-- We represent a "subset" as a predicate α → Prop
abbrev Pred (α : Type) := α → Prop

class CompleteLattice (α : Type) extends Lattice α where
  sSup : Pred α → α    -- supremum of a set  (⨆)
  sInf : Pred α → α    -- infimum of a set   (⨅)
  le_sSup : ∀ (s : Pred α) (a : α), s a → a ≤ sSup s
  sSup_le : ∀ (s : Pred α) (u : α), (∀ a, s a → a ≤ u) → sSup s ≤ u
  sInf_le : ∀ (s : Pred α) (a : α), s a → sInf s ≤ a
  le_sInf : ∀ (s : Pred α) (l : α), (∀ a, s a → l ≤ a) → l ≤ sInf s

end OrderBook
```

The powerset lattice is the canonical example of a complete lattice: $\bigvee \mathcal{F} = \bigcup \mathcal{F}$ and $\bigwedge \mathcal{F} = \bigcap \mathcal{F}$.

```
-- Set α is a complete lattice
instance {α : Type} : OrderBook.CompleteLattice (Set α) where
  -- (inheriting the Lattice instance from Chapter 3)
  sSup := fun F a => ∃ s : Set α, F s ∧ s a    -- union of a family
  sInf := fun F a => ∀ s : Set α, F s → s a    -- intersection of a family
  le_sSup := fun F s hs a ha => ⟨s, hs, ha⟩
  sSup_le := fun F u hu a ⟨s, hs, ha⟩ => hu s hs a ha
  sInf_le := fun F s hs a ha => ha s hs
  le_sInf := fun F l hl a ha s hs => hl s hs a ha
  -- (le, inf, sup fields inherited)
  le          := Set.subset
  le_refl     := fun s a h => h
  le_antisymm := fun s t h1 h2 => funext fun a => propext ⟨h1 a, h2 a⟩
  le_trans    := fun s t u h1 h2 a ha => h2 a (h1 a ha)
  inf         := Set.inter; sup := Set.union
  inf_le_left  := fun s t a ⟨hs,_⟩ => hs
  inf_le_right := fun s t a ⟨_,ht⟩ => ht
  le_inf       := fun s t u hst hsu a ha => ⟨hst a ha, hsu a ha⟩
  sup_le_left  := fun s t a hs => Or.inl hs
  sup_le_right := fun s t a ht => Or.inr ht
  le_sup       := fun s t u hsu htu a h => h.elim (hsu a) (htu a)
```

4.2 The Knaster–Tarski Fixed-Point Theorem

Knaster–Tarski (1955)

Let $(L, \leq)$ be a complete lattice and $f : L \to L$ a monotone function. Then f has a *least fixed point* $\mu f = \bigwedge \{x \mid f(x) \leq x\}$ and a *greatest fixed point* $\nu f = \bigvee \{x \mid x \leq f(x)\}$.

Proof. Let $P = \{x \in L \mid f(x) \leq x\}$ (the set of *pre-fixed points*) and let $m = \bigwedge P$.

Step 1: $f(m) \leq m$. For every $x \in P$ we have $m \leq x$, so $f(m) \leq f(x) \leq x$ by monotonicity. Hence $f(m)$ is a lower bound of P, so $f(m) \leq m$.

Step 2: $m \leq f(m)$. From Step 1, $f(f(m)) \leq f(m)$ (applying f to $f(m) \leq m$ and using monotonicity gives $f(f(m)) \leq f(m)$, so $f(m) \in P$). Since m is the infimum of P, we have $m \leq f(m)$.

Conclusion: $f(m) = m$, so m is a fixed point. Any other fixed point x^* satisfies $f(x^*) = x^* \leq x^*$, so $x^* \in P$, hence $m \leq x^*$. So m is the *least* fixed point. □

```
namespace OrderBook
open CompleteLattice PartialOrder

-- Knaster-Tarski: least fixed point of a monotone function
-- on a complete lattice
theorem knaster_tarski {α : Type} [CompleteLattice α]
    (f : α → α) (hf : Monotone f) :
    ∃ (lfp : α), f lfp = lfp ∧
      ∀ (x : α), f x = x → lfp ≤ x := by
  -- Define P = {x | f(x) ≤ x} and m = ⊓ P
  let P : Pred α := fun x => f x ≤ x
  let m := sInf P
  -- Step 1: f(m) ≤ m
  have step1 : f m ≤ m := by
    apply le_sInf
    intro x hx
    -- m ≤ x, so f(m) ≤ f(x) ≤ x
    exact le_trans (hf.map_le m x (sInf_le P x hx)) hx
  -- Step 2: m ≤ f(m)  (f(m) ∈ P since f(f(m)) ≤ f(m))
  have step2 : m ≤ f m := by
    apply sInf_le
    exact hf.map_le _ _ step1
  -- Conclude m is a fixed point
  have fixed : f m = m := le_antisymm step1 step2
  refine ⟨m, fixed, ?_⟩
  -- Minimality
  intro x hx
  apply sInf_le
  show f x ≤ x
  exact le_of_eq hx
end OrderBook
```

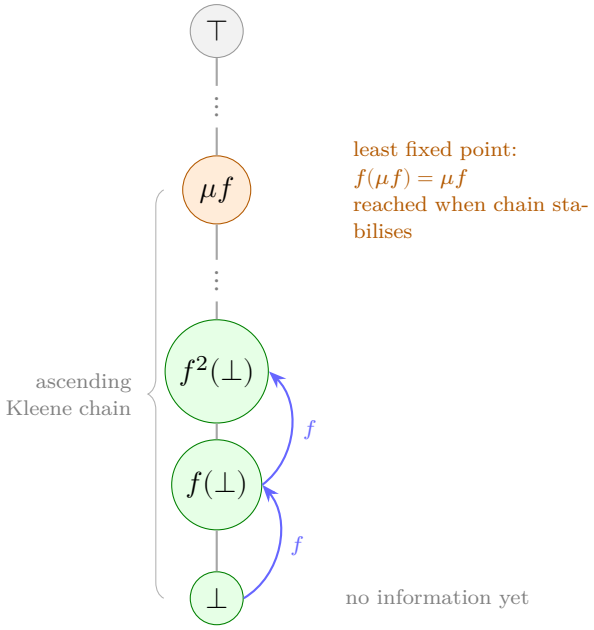


Figure 4.1: The ascending Kleene chain starting from $\bot$ under a monotone function f on a complete lattice. Each application of f moves upward. If the lattice has no infinite strictly ascending chains (the ascending chain condition), the sequence stabilises at the least fixed point μf. This is exactly what a propagator network computes.

4.3 Why Fixed Points Matter

The Knaster–Tarski theorem is not just elegant theory; it is the backbone of:

- **Dataflow analysis**: the least fixed point of a transfer function gives the most precise (and sound) analysis result.
- **Denotational semantics**: the meaning of a recursive program is the least fixed point of a semantic functional.
- **Propagator networks**: running a network to quiescence is exactly computing a fixed point of the combined propagation function.

We will return to this connection in Chapter 6 when we prove that propagator network execution terminates and is correct.

4.4 Exercises

Exercise 4.1 — Greatest fixed point

Using the Knaster–Tarski proof as a template, prove the existence of the *greatest* fixed point $\nu f = \bigvee\{x \mid x \leq f(x)\}$.

Exercise 4.2 — Ascending Kleene chain

For a complete lattice with a bottom element $\bot$ and a monotone f, the sequence $\bot \leq f(\bot) \leq f^2(\bot) \leq \cdots$ is called the *ascending Kleene chain*. If the lattice satisfies the *ascending chain condition* (every ascending chain eventually stabilises), this chain reaches μf in finitely many steps. Prove in Lean 4 that each element of this sequence is $\leq$ the next.

Part II

Propagator Networks

Chapter 5

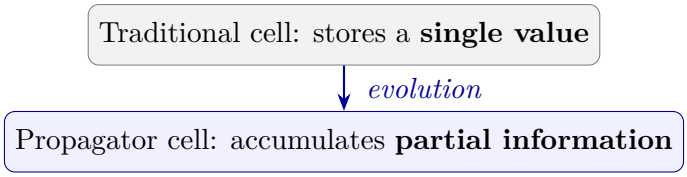
The Propagator Model

> *"A propagator network is a collection of machines that compute values for cells by combining information from other cells."*
> — *G.J. Sussman & A. Radul,* The Art of the Propagator*, 2009*

We now have all the mathematical machinery we need. This chapter explains the propagator model from first principles and makes precise why lattices are not merely convenient but *necessary*.

5.1 Cells: Accumulating Partial Information

The central insight of propagator networks is a shift in perspective:



A cell does not *store a value*; it stores *everything we know so far* about a value. New information can arrive from multiple sources and be merged in. The key requirement is that information can only *increase*: once we know something, we cannot un-know it.

This is precisely the structure of a *join-semilattice with a bottom element*: the bottom represents "we know nothing", and merging two pieces of information is the join ($\sqcup$).

Definition — Information Order

A *bounded join-semilattice* is a partial order $(J, \leq)$ equipped with a join $\sqcup$ and a bottom $\bot$ such that:

- $a \sqcup b$ is the least upper bound of a and b,
- $\bot \leq a$ for all a,
- $a \leq b$ is read as "b contains at least as much information as a".

We call this the *information order*.

```
1 namespace Propagators
2
```

```
class BoundedJoinSemilattice (α : Type) extends OrderBook.PartialOrder α where
  sup : α → α → α
  bot : α
  le_sup_left  : ∀ (a b : α), a ≤ sup a b
  le_sup_right : ∀ (a b : α), b ≤ sup a b
  sup_le       : ∀ (a b c : α), a ≤ c → b ≤ c → sup a b ≤ c
  bot_le       : ∀ (a : α), bot ≤ a

-- Convenient notation
infixl:65 " ⊔ " => BoundedJoinSemilattice.sup
notation    "⊥"  => BoundedJoinSemilattice.bot

end Propagators
```

5.2 Why Information Can Only Increase

Suppose a propagator computes that "x is between 3 and 7". Later another propagator deduces that "x is between 5 and 12". Combining these we know "x is between 5 and 7". Notice: the combined knowledge is *strictly more specific* than either input.

If we allowed information to *decrease* (e.g., forgetting the $[3, 7]$ constraint and keeping only $[5, 12]$), we would lose soundness: our cell might claim values that the original propagators had ruled out.

Monotonicity Ensures Soundness

In a propagator network over a bounded join-semilattice, if every propagator is monotone, then the network is *sound*: the final cell values are consistent with all constraints.

Intuitively: since each step can only add information (move up in the lattice), the final state contains at least as much information as any intermediate state. Nothing is lost; nothing false is added (assuming correct propagators).

5.3 Propagators as Monotone Functions

Definition — Propagator

Given cells with values in $(J, \leq)$, a *propagator* is a monotone function $f : J^n \to J$ (reading from n input cells and writing to one output cell by merging the result).

Writing is done by merging: the output cell c receives $c \leftarrow c \sqcup f(\text{inputs})$. Since $\sqcup$ is the least upper bound, this can only increase c.

```
namespace Propagators

-- A unary propagator
structure Propagator1 (α β : Type)
    [BoundedJoinSemilattice α]
    [BoundedJoinSemilattice β] where
  fn       : α → β
  monotone : ∀ (a b : α), a ≤ b →
    BoundedJoinSemilattice.sup (fn a) (fn b) = fn b

-- A binary propagator
structure Propagator2 (α β γ : Type)
    [BoundedJoinSemilattice α]
    [BoundedJoinSemilattice β]
    [BoundedJoinSemilattice γ] where
  fn       : α → β → γ
  monotone : ∀ (a1 a2 : α) (b1 b2 : β),
    a1 ≤ a2 → b1 ≤ b2 →
    BoundedJoinSemilattice.sup (fn a1 b1) (fn a2 b2) = fn a2 b2

end Propagators
```

5.4 Networks, Scheduling, and Quiescence

A *propagator network* is a directed graph where nodes are cells and edges are propagators. Execution proceeds as follows:

1. Initialise all cells to $\bot$.
2. Place all propagators on a *work queue*.
3. While the queue is non-empty:
   (a) Take a propagator p from the queue.
   (b) Compute the new value $v = p(\text{input cells})$.
   (c) Merge v into the output cell: $c \leftarrow c \sqcup v$.
   (d) If c changed, add all propagators that read from c back to the queue.
4. The network has reached *quiescence* when the queue is empty.

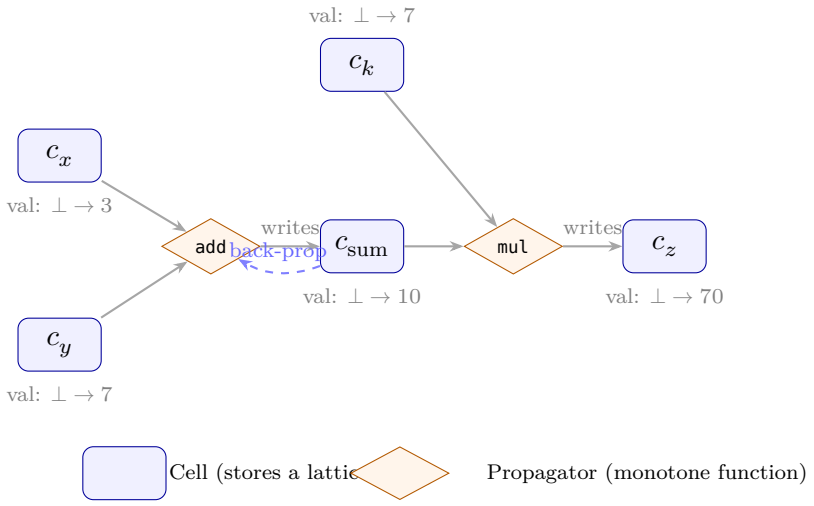


Figure 5.1: A small propagator network computing $x+y = \text{sum}$ and $\text{sum} \times k = z$ (with $x = 3$, $y = 7$, $k = 7$, so $\text{sum} = 10$ and $z = 70$). Cells (rounded rectangles) hold lattice values, initialised to $\bot$. Propagators (diamonds) read from their input cells and write to their output cell by merging with $\sqcup$. The dashed arrow shows back-propagation: when c_{sum} is updated the `add` propagator is re-queued to potentially narrow c_x or c_y.

Convergence

If the lattice satisfies the *ascending chain condition* (every strictly ascending sequence is finite), then the network reaches quiescence in finite time.

This is immediate: each cell value is non-decreasing and the chain condition guarantees it cannot increase indefinitely.

For our capstone projects, the domain lattice is always finite, so convergence is guaranteed.

5.5 The Fundamental Role of the Lattice

Let us be precise about why we need a lattice rather than merely some arbitrary type.

- **Join ($\sqcup$) for merging.** When two propagators independently derive information about the same cell, we need to combine their results. The join gives the *least informative* value that is *at least as informative* as both. No information is lost; no information is invented.
- **Bottom ($\bot$) for initialisation.** Before any propagator has run, a cell contains *no information*. The bottom element represents this "empty" state.
- **Partial order ($\leq$) for monitoring change.** We must know whether a cell's value actually changed after merging, to decide whether to re-schedule dependent propagators. The partial order provides this: if $c \sqcup v = c$ (the new value adds no information), nothing changed.

- **Antisymmetry for termination.** If a cell's value can only go up and the lattice is finite, the antisymmetry of $\leq$ ensures each value is visited at most once.

Any type that fails to be a join-semilattice with $\bot$ is simply not a valid cell domain for a propagator network. The lattice structure is not a convenience — it is a prerequisite.

Chapter 6

Building a Propagator Network in Lean 4

Theory in hand, we now build a complete, runnable propagator network in Lean 4, starting from our typeclass definitions and ending with a working scheduler.

6.1 Mutable Cells

Lean 4 has an `IO` monad with mutable references (`IO.Ref`). We use these to implement cells.

```
namespace Propagators

-- A Cell wraps an IO.Ref and a type with a BoundedJoinSemilattice
structure Cell (α : Type) where
  ref      : IO.Ref α

variable {α : Type} [BoundedJoinSemilattice α] [DecidableEq α]

-- Create a new cell initialised to ⊥
def Cell.new : IO (Cell α) := do
  let r ← IO.mkRef (BoundedJoinSemilattice.bot (α := α))
  return { ref := r }

-- Read the current value
def Cell.read (c : Cell α) : IO α := c.ref.get

-- Attempt to write new information.
-- Returns true if the cell's value increased.
def Cell.write (c : Cell α) (v : α) : IO Bool := do
  let old ← c.ref.get
  let merged := old ⊔ v
  if merged == old then
    return false          -- no new information
  else
    c.ref.set merged
```

```
    return true           -- cell updated, re-schedule dependents

end Propagators
```

We require `DecidableEq α` to compare `merged == old`. For the finite lattices we use in our capstone projects this is always available.

6.2 The Network and Scheduler

```
namespace Propagators

-- A propagator step is an IO action that returns
-- true if it caused any cell to change.
abbrev PropStep := IO Bool

-- Run a list of propagators to fixpoint.
-- Each pass re-runs all propagators; we stop when
-- a full pass causes no changes.
def runToFixpoint (steps : Array PropStep) : IO Unit := do
  let mut changed := true
  while changed do
    changed := false
    for step in steps do
      let c ← step
      if c then changed := true

-- A smarter scheduler that only re-runs propagators
-- whose inputs changed (using a dependency queue).
structure Network where
  steps    : Array PropStep
  -- In a production implementation, steps would be indexed
  -- and cells would hold lists of dependent propagator IDs.
  -- For clarity we use the simple "run all" scheduler here.

def Network.run (n : Network) : IO Unit :=
  runToFixpoint n.steps

end Propagators
```

6.3 Building Propagators: Numeric Example

The simplest useful cell domain is the *flat lattice*: values are either completely unknown, exactly determined, or contradictory.

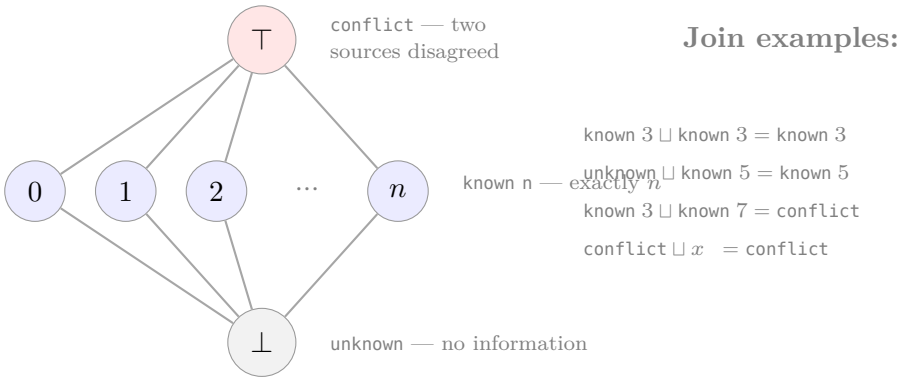


Figure 6.1: The flat natural number lattice `FlatNat`. The bottom $\bot$ represents "no information yet". Each concrete value n is incomparable with all other concrete values — knowing the answer is 3 tells you nothing about whether it is 7. The top $\top$ represents a contradiction. This lattice has height 2, so any cell stabilises after at most two updates.

```
namespace Propagators

inductive FlatNat where
  | unknown      : FlatNat          -- ⊥: no information
  | known (n : Nat) : FlatNat       -- exactly n
  | conflict     : FlatNat          -- ⊤: contradiction

instance : OrderBook.PartialOrder FlatNat where
  le a b := match a, b with
    | .unknown, _       => True     -- unknown ≤ everything
    | _, .conflict      => True     -- everything ≤ conflict
    | .known m, .known n => m = n   -- known values only ≤ themselves
    | .conflict, .unknown => False
    | .conflict, .known _ => False
    | .known _, .unknown  => False
  le_refl  := by intro a; cases a <;> simp
  le_antisymm := by
    intro a b hab hba
    cases a <;> cases b <;> simp_all
  le_trans := by
    intro a b c hab hbc
    cases a <;> cases b <;> cases c <;> simp_all

instance : BoundedJoinSemilattice FlatNat where
```

```
  le          := OrderBook.PartialOrder.le
  le_refl     := OrderBook.PartialOrder.le_refl
  le_antisymm := OrderBook.PartialOrder.le_antisymm
  le_trans    := OrderBook.PartialOrder.le_trans
  bot         := .unknown
  sup a b := match a, b with
    | .unknown, x      => x
    | x, .unknown      => x
    | .known m, .known n => if m = n then .known m else .conflict
    | _, _             => .conflict
  bot_le      := by intro a; cases a <;> simp [OrderBook.PartialOrder.le]
  sup_le_left  := by intro a b; cases a <;> cases b <;> simp_all
    ↪ [OrderBook.PartialOrder.le]
  sup_le_right := by intro a b; cases a <;> cases b <;> simp_all
    ↪ [OrderBook.PartialOrder.le]
  le_sup       := by
    intro a b c hac hbc
    cases a <;> cases b <;> cases c <;> simp_all [OrderBook.PartialOrder.le]

deriving instance DecidableEq for FlatNat

end Propagators
```

6.4 A Complete Running Example

Example — Arithmetic propagation

We know: $x + y = 10$ and $x = 3$. A propagator should deduce $y = 7$.

```
namespace Propagators

open FlatNat

-- Addition propagator: given x and y cells, write x+y to sum cell
def addProp (cx cy csum : Cell FlatNat) : PropStep := do
  let x ← cx.read
  let y ← cy.read
  match x, y with
  | .known a, .known b => csum.write (.known (a + b))
  | _, _ => return false

-- Subtraction (used to deduce y = sum - x)
def subProp (csum cx cy : Cell FlatNat) : PropStep := do
```

```
  let s ← csum.read
  let x ← cx.read
  match s, x with
  | .known total, .known a =>
    if total ≥ a
    then cy.write (.known (total - a))
    else cy.write .conflict
  | _, _ => return false

def exampleNetwork : IO Unit := do
  let cx   ← Cell.new (α := FlatNat)
  let cy   ← Cell.new (α := FlatNat)
  let csum ← Cell.new (α := FlatNat)

  -- Seed known facts
  let _ ← cx.write (.known 3)
  let _ ← csum.write (.known 10)

  let net : Network := {
    steps := #[
      addProp cx cy csum,
      subProp csum cx cy,
      subProp csum cy cx
    ]
  }

  net.run

  let yVal ← cy.read
  match yVal with
  | .known n => IO.println s!"y = {n}"   -- prints: y = 7
  | .unknown  => IO.println "y is unknown"
  | .conflict => IO.println "contradiction!"

end Propagators
```

6.5 Why the Lattice Is Load-Bearing Here

Notice several things:

- `Cell.write` can be called multiple times from different propagators without coordination. The join ensures the final value is consistent with all of them.
- Running `addProp` when either input is `unknown` correctly does nothing (returns `false`), waiting for more information.

- The `conflict` element propagates correctly: if two propagators derive incompatible values, the join produces `conflict`, which then propagates to any cell that depends on this one.
- The whole system terminates because `FlatNat` has height 2: after at most two increases, any cell stabilises.

6.6 Exercises

Exercise 6.1 — Three-cell constraint

Add a `mulProp` propagator for $x \times y = z$ with the `FlatNat` lattice. Build a network with $x \times y = 12$, $x = 3$, and verify it deduces $y = 4$.

Exercise 6.2 — Conflict detection

Seed a network with $x = 3$ from one source and $x = 5$ from another. Verify that `cx.read` returns `.conflict`.

Chapter 7

The Interval Lattice

Flat lattices are powerful but coarse: a cell either knows the exact value or knows nothing. *Interval lattices* let us represent partial numeric knowledge as a range, enabling *bound propagation* — the engine behind many real-world constraint solvers.

7.1 Intervals as Partial Information

An interval $[l, h]$ says: "the true value lies between l and h". The wider the interval, the less we know. The narrowest possible interval is a single point $[n, n]$, meaning we know the value exactly.

Definition — Interval Lattice

Let the domain be extended integers $\mathbb{Z}_\infty = \mathbb{Z} \cup \{-\infty, +\infty\}$. Define:

- An *interval* is either $\bot$ (no values possible, contradiction) or $[l, h]$ with $l \leq h$.
- The *information order*: $[l_1, h_1] \leq [l_2, h_2] \iff l_2 \leq l_1 \wedge h_1 \leq h_2$ (a narrower interval contains more information).
- $\bot \leq [l, h]$ for all intervals.
- The *join* (merge two pieces of information): $[l_1, h_1] \sqcup [l_2, h_2] = [\min(l_1, l_2), \max(h_1, h_2)]$ (the smallest interval containing both).
- $\bot$ is the identity for $\sqcup$.

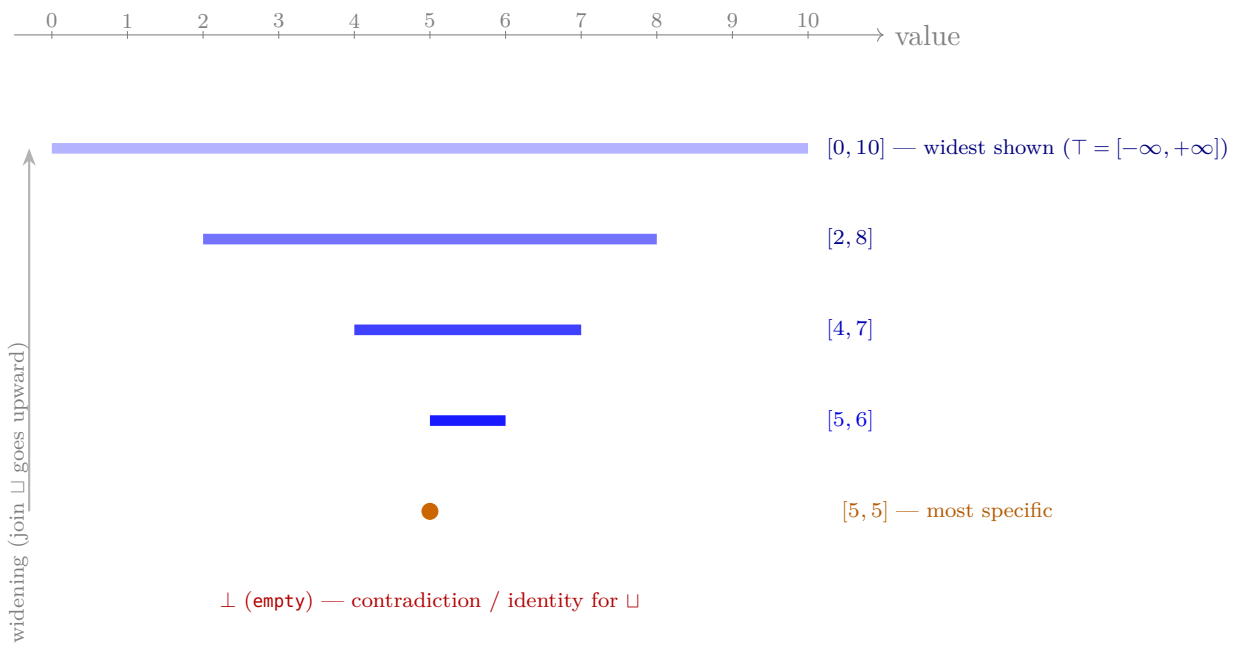


Figure 7.1: The interval lattice order as used in this book's propagators. ⊥ (`empty`) sits at the bottom as the join identity: $\bot \sqcup x = x$. Going upward, intervals widen. The join (`Interval.sup`) of two ranges is their *hull* (smallest interval containing both), which is at least as wide as either input and therefore weakly higher in the order. The meet (`Interval.intersect`) is the intersection, which is narrower and thus lower. The widest possible interval $[-\infty, +\infty]$ is $\top$.

```
namespace Intervals

inductive Interval where
  | empty              : Interval   -- ⊥
  | range (lo hi : Int) (h : lo ≤ hi) : Interval

-- Information order: narrower = more informative = higher
def Interval.le (a b : Interval) : Prop :=
  match a, b with
  | .empty, _                => True
  | .range _ _ _, .empty     => False
  | .range l1 h1 _, .range l2 h2 _ => l2 ≤ l1 ∧ h1 ≤ h2

-- Join: widen to contain both intervals
def Interval.sup (a b : Interval) : Interval :=
  match a, b with
  | .empty, x => x
  | x, .empty => x
  | .range l1 h1 _, .range l2 h2 _ =>
    let l := min l1 l2
    let h := max h1 h2
```

```
    .range l h (by omega)

instance : OrderBook.PartialOrder Interval where
  le          := Interval.le
  le_refl     := by intro a; cases a <;> simp [Interval.le]
  le_antisymm := by
    intro a b hab hba
    cases a <;> cases b <;>
      simp [Interval.le] at * <;> omega
  le_trans    := by
    intro a b c hab hbc
    cases a <;> cases b <;> cases c <;>
      simp [Interval.le] at * <;> omega

end Intervals
```

7.2 Interval Arithmetic Propagators

With the interval lattice, we can implement real *bound propagation*.

```
namespace Intervals

-- Add two intervals
def Interval.add (a b : Interval) : Interval :=
  match a, b with
  | .empty, _ | _, .empty => .empty
  | .range l1 h1 _, .range l2 h2 _ =>
    .range (l1 + l2) (h1 + h2) (by omega)

-- Subtract: a - b = [l1-h2, h1-l2]
def Interval.sub (a b : Interval) : Interval :=
  match a, b with
  | .empty, _ | _, .empty => .empty
  | .range l1 h1 _, .range l2 h2 _ =>
    .range (l1 - h2) (h1 - l2) (by omega)

-- Intersect (meet in the information order — more precise)
def Interval.intersect (a b : Interval) : Interval :=
  match a, b with
  | .empty, _ | _, .empty => .empty
  | .range l1 h1 _, .range l2 h2 _ =>
    let l := max l1 l2
    let h := min h1 h2
    if h_le : l ≤ h then .range l h h_le else .empty
```

```
25
26 end Intervals
```

7.3 A Worked Example: Bound Propagation

Example — $x + y = 10$, $x \in [2, 8]$

What is the tightest interval for y?
From $x + y = 10$ we get $y = 10 - x$. Since $x \in [2, 8]$, we have $y \in [10 - 8, 10 - 2] = [2, 8]$. The propagator should derive this automatically.

```
namespace Intervals

-- Propagator: sum = x + y (bidirectional: also narrows x and y)
def addConstraint
    (cx cy csum : Propagators.Cell Interval) : Array Propagators.PropStep :=
  #[
    -- Forward: sum ← x + y
    do
      let x ← cx.read; let y ← cy.read
      csum.write (Interval.add x y),
    -- Backward: x ← sum - y
    do
      let s ← csum.read; let y ← cy.read
      cx.write (Interval.sub s y),
    -- Backward: y ← sum - x
    do
      let s ← csum.read; let x ← cx.read
      cy.write (Interval.sub s x)
  ]

def intervalExample : IO Unit := do
  let cx   ← Propagators.Cell.new (α := Interval)
  let cy   ← Propagators.Cell.new (α := Interval)
  let csum ← Propagators.Cell.new (α := Interval)

  -- x ∈ [2, 8], sum = 10
  let _ ← cx.write (.range 2 8 (by omega))
  let _ ← csum.write (.range 10 10 (by omega))

  let steps := addConstraint cx cy csum
  Propagators.runToFixpoint steps

```

```
  let yVal ← cy.read
  match yVal with
  | .empty        => IO.println "y: impossible"
  | .range l h _  => IO.println s!"y ∈ [{l}, {h}]"   -- y ∈ [2, 8]

end Intervals
```

The join in action. When the backward propagator writes $[2, 8]$ to c_y and c_y already holds $\bot$, the join $\bot \sqcup [2, 8] = [2, 8]$ correctly updates the cell. On the next pass, the forward propagator reads $x = [2, 8]$ and $y = [2, 8]$, computes add $= [4, 16]$, and writes to c_{sum}. Since $[10, 10] \sqcup [4, 16] = [4, 16]$ would *widen* the constraint (lose information), we could use `intersect` here for a more powerful solver. We leave this as Exercise 7.1.

7.4 Exercises

Exercise 7.1 — Intersection for tighter bounds

Replace `Interval.sup` (widening join) with `Interval.intersect` (narrowing meet) in the cell write operation for the sum constraint. Explain why the information order must be *reversed* when you switch from widening to narrowing. What does this say about the dual lattice?

Exercise 7.2 — Multiplication intervals

Implement `Interval.mul`. Be careful: multiplying negative intervals requires considering all four corner products.

Part III

Capstone Projects

Chapter 8

Capstone I: A Sudoku Solver

Sudoku is a constraint satisfaction problem: each of 81 cells must receive a digit 1–9 subject to row, column, and box uniqueness constraints. Propagator networks handle this beautifully. In this chapter we build a complete solver using nothing but lattice-ordered propagation.

8.1 The Domain Lattice

Each Sudoku cell can be modelled as a set of *candidate digits*. Initially every cell can hold any of $\{1, \dots, 9\}$. Constraints eliminate candidates.

Definition — Candidate Set Lattice

Let $D = \{1, \dots, 9\}$. The *candidate-set lattice* is the powerset $\mathcal{P}(D)$ ordered by *reverse inclusion*:

$$S \leq T \iff T \subseteq S.$$

The bottom $\bot = D$ (no information: any digit possible), the top $\top = \emptyset$ (contradiction: no digit possible). The join is intersection: $S \sqcup T = S \cap T$ (the smallest set of candidates consistent with both sources).

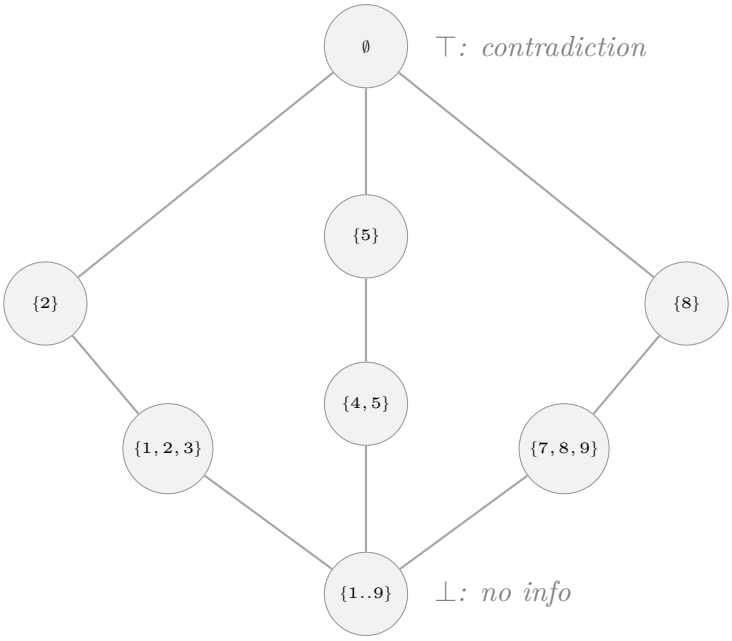


Figure 8.1: A fragment of the candidate-set lattice (reverse inclusion order).

This is precisely the *dual* powerset lattice from Exercise 1.4!

```
namespace Sudoku

-- Digits 1..9, represented as Fin 9 (0..8 internally, displayed as 1..9)
abbrev Digit := Fin 9

-- A candidate set: which digits are still possible for this cell
-- We represent it as a Bool array of length 9
structure CandidateSet where
  present : Fin 9 → Bool

def CandidateSet.full : CandidateSet :=
  { present := fun _ => true }

def CandidateSet.empty : CandidateSet :=
  { present := fun _ => false }

def CandidateSet.singleton (d : Digit) : CandidateSet :=
  { present := fun i => i == d }

def CandidateSet.remove (s : CandidateSet) (d : Digit) : CandidateSet :=
  { present := fun i => s.present i && !(i == d) }

def CandidateSet.inter (s t : CandidateSet) : CandidateSet :=
  { present := fun i => s.present i && t.present i }

```

```
def CandidateSet.union (s t : CandidateSet) : CandidateSet :=
  { present := fun i => s.present i || t.present i }

-- The information order: s ≤ t iff t ⊆ s (t is more specific)
def CandidateSet.le (s t : CandidateSet) : Prop :=
  ∀ i : Digit, t.present i = true → s.present i = true

instance : OrderBook.PartialOrder CandidateSet where
  le          := CandidateSet.le
  le_refl     := fun s i h => h
  le_antisymm := fun s t h1 h2 =>
    show s = t from by
      cases s; cases t
      congr 1; funext i
      cases hi : t.present i
      · simp [CandidateSet.le] at h1
        cases hs : s.present i
        · rfl
        · exact absurd (h1 i hs) (by simp [hi])
      · exact h2 i hi
  le_trans    := fun s t u hst htu i hi => hst i (htu i hi)

-- Join = intersection (combines constraints)
instance : Propagators.BoundedJoinSemilattice CandidateSet where
  le          := CandidateSet.le
  le_refl     := fun s i h => h
  le_antisymm := OrderBook.PartialOrder.le_antisymm (α := CandidateSet)
  le_trans    := OrderBook.PartialOrder.le_trans (α := CandidateSet)
  bot         := CandidateSet.full
  sup         := CandidateSet.inter    -- ⊔ = ∩ in information order!
  bot_le      := fun s i _ => rfl      -- full ≤ s since s ⊆ full
  sup_le_left  := fun s t i h => (Bool.and_eq_true.mp h).1
  sup_le_right := fun s t i h => (Bool.and_eq_true.mp h).2
  le_sup       := fun s t u hsu htu i h =>
    Bool.and_eq_true.mpr ⟨hsu i h, htu i h⟩

instance : DecidableEq CandidateSet :=
  fun s t =>
    if h : ∀ i, s.present i = t.present i
    then isTrue  (by cases s; cases t; congr 1; funext i; exact h i)
    else isFalse (fun heq => h (fun i => by rw [heq]))

end Sudoku
```

8.2 Representing the Grid

```
namespace Sudoku

-- 81 cells indexed by row and column (both Fin 9)
abbrev Grid := Fin 9 → Fin 9 → Propagators.Cell CandidateSet

def makeGrid : IO Grid := do
  let mut rows : Array (Array (Propagators.Cell CandidateSet)) := #[]
  for _ in [:9] do
    let mut row : Array (Propagators.Cell CandidateSet) := #[]
    for _ in [:9] do
      let c ← Propagators.Cell.new (α := CandidateSet)
      row := row.push c
    rows := rows.push row
  -- Convert to function (safe because sizes match)
  return fun r c => rows[r.val]![c.val]!

-- Seed a given digit into a cell
def seedCell (grid : Grid) (r c : Fin 9) (d : Digit) : IO Unit := do
  let _ ← (grid r c).write (CandidateSet.singleton d)

end Sudoku
```

8.3 Constraint Propagators

Sudoku has three families of constraints: row, column, and 3×3 box uniqueness. Each is expressed as a propagator that, given a cell known to hold digit d, removes d from all peers.

```
namespace Sudoku

-- If cell (r,c) is determined to be d, remove d from all other cells in row r
def rowPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
  let cands ← (grid r c).read
  -- Check if this cell is a singleton
  let det : Option Digit := Id.run do
    let mut found : Option Digit := none
    let mut count := 0
    for i in [:9] do
      let i' : Digit := ⟨i, by omega⟩
      if cands.present i' then
        found := some i'; count := count + 1
```

```
    if count = 1 then found else none
  match det with
  | none   => return false
  | some d =>
    let mut changed := false
    for j in [:9] do
      let j' : Fin 9 := ⟨j, by omega⟩
      if j' ≠ c then
        let removed := { present := fun i => (grid r j').read.present i && !(i
          ↪  == d) }
        -- We re-read and merge
        let old ← (grid r j').read
        let upd := CandidateSet.inter old { present := fun i => !(i == d) }
        let c ← (grid r j').write upd
        if c then changed := true
    return changed

-- Analogous: column propagator
def colPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
  let cands ← (grid r c).read
  let det : Option Digit := Id.run do
    let mut found : Option Digit := none
    let mut count := 0
    for i in [:9] do
      let i' : Digit := ⟨i, by omega⟩
      if cands.present i' then found := some i'; count := count + 1
    if count = 1 then found else none
  match det with
  | none => return false
  | some d =>
    let mut changed := false
    for i in [:9] do
      let i' : Fin 9 := ⟨i, by omega⟩
      if i' ≠ r then
        let old ← (grid i' c).read
        let upd := CandidateSet.inter old { present := fun k => !(k == d) }
        let ch ← (grid i' c).write upd
        if ch then changed := true
    return changed

-- Box propagator (which 3×3 box contains (r,c)?)
def boxPropagator (grid : Grid) (r c : Fin 9) : Propagators.PropStep := do
  let cands ← (grid r c).read
  let det : Option Digit := Id.run do
    let mut found : Option Digit := none; let mut count := 0
    for i in [:9] do
```

```
      let i' : Digit := ⟨i, by omega⟩
      if cands.present i' then found := some i'; count := count + 1
    if count = 1 then found else none
  match det with
  | none => return false
  | some d =>
    let boxR := r.val / 3 * 3
    let boxC := c.val / 3 * 3
    let mut changed := false
    for dr in [:3] do
      for dc in [:3] do
        let r' : Fin 9 := ⟨boxR + dr, by omega⟩
        let c' : Fin 9 := ⟨boxC + dc, by omega⟩
        if !(r' = r && c' = c) then
          let old ← (grid r' c').read
          let upd := CandidateSet.inter old { present := fun k => !(k == d) }
          let ch ← (grid r' c').write upd
          if ch then changed := true
    return changed

end Sudoku
```

8.4 Assembling and Running the Solver

```
namespace Sudoku

def buildNetwork (grid : Grid) : Array Propagators.PropStep := Id.run do
  let mut steps : Array Propagators.PropStep := #[]
  for r in [:9] do
    for c in [:9] do
      let r' : Fin 9 := ⟨r, by omega⟩
      let c' : Fin 9 := ⟨c, by omega⟩
      steps := steps.push (rowPropagator grid r' c')
      steps := steps.push (colPropagator grid r' c')
      steps := steps.push (boxPropagator grid r' c')
  return steps

-- Print the grid (showing the unique digit or '?' for unsolved cells)
def printGrid (grid : Grid) : IO Unit := do
  for r in [:9] do
    let mut row := ""
    for c in [:9] do
      let r' : Fin 9 := ⟨r, by omega⟩
```

```
      let c' : Fin 9 := ⟨c, by omega⟩
      let cands ← (grid r' c').read
      let mut digit := '?'
      let mut count := 0
      for i in [:9] do
        let i' : Digit := ⟨i, by omega⟩
        if cands.present i' then digit := Char.ofNat (i + 49); count := count
          ↪ + 1
      row := row ++ (if count = 1 then s!"{digit}" else "?")
      if c % 3 == 2 && c < 8 then row := row ++ "|"
    IO.println row
    if r % 3 == 2 && r < 8 then IO.println "-----------"

-- Example: a simple Sudoku puzzle
-- (0 = unknown, digits are given clues)
def examplePuzzle : Array (Array Nat) := #[
  #[5, 3, 0, 0, 7, 0, 0, 0, 0],
  #[6, 0, 0, 1, 9, 5, 0, 0, 0],
  #[0, 9, 8, 0, 0, 0, 0, 6, 0],
  #[8, 0, 0, 0, 6, 0, 0, 0, 3],
  #[4, 0, 0, 8, 0, 3, 0, 0, 1],
  #[7, 0, 0, 0, 2, 0, 0, 0, 6],
  #[0, 6, 0, 0, 0, 0, 2, 8, 0],
  #[0, 0, 0, 4, 1, 9, 0, 0, 5],
  #[0, 0, 0, 0, 8, 0, 0, 7, 9]
]

def solveSudoku : IO Unit := do
  let grid ← makeGrid
  -- Seed known digits
  for r in [:9] do
    for c in [:9] do
      let d := examplePuzzle[r]![c]!
      if d ≠ 0 then
        let r' : Fin 9 := ⟨r, by omega⟩
        let c' : Fin 9 := ⟨c, by omega⟩
        seedCell grid r' c' ⟨d - 1, by omega⟩
  -- Build and run the propagator network
  let steps := buildNetwork grid
  Propagators.runToFixpoint steps
  -- Print the result
  IO.println "Solution:"
  printGrid grid

end Sudoku
```

8.5 Why This Works: The Lattice Perspective

- Each cell starts at $\bot = \{1, \ldots, 9\}$: no information.
- Every propagator only writes using `CandidateSet.inter` (the join in the information order), which can only *reduce* candidate sets — i.e., move *up* in the information order.
- The network terminates because the candidate-set lattice is finite and the cells only move upward.
- Reaching $\top = \emptyset$ in any cell signals a contradiction.
- A solved cell is one that has reached a singleton $\{d\}$.

The elegant insight: *solving Sudoku is computing the least fixed point of the combined propagation function*, where the partial order is the information order on candidate sets. Knaster–Tarski guarantees it exists; our scheduler computes it.

Exercise 8.1 — Naked pair

A *naked pair* occurs when two cells in the same unit both have exactly the same two candidates $\{a, b\}$. Then a and b can be removed from all other cells in that unit. Implement this as an additional propagator.

Exercise 8.2 — Contradiction detection

Add a `contradictionProp` that checks every cell; if any cell reaches $\top = \emptyset$, print a diagnostic and halt. Why is this detection sound (i.e., why can't a cell reach $\emptyset$ unless the puzzle is truly unsatisfiable)?

Chapter 9

Capstone II: Type Inference via Propagation

Our second capstone takes us into programming language theory. Type inference — the process of automatically deducing types of expressions — is naturally formulated as constraint propagation on a type lattice.

9.1 A Simple Expression Language

We work with a fragment of the lambda calculus extended with integers and booleans.

```
namespace TypeInfer

-- Expressions
inductive Expr where
  | intLit  (n : Int)        : Expr
  | boolLit (b : Bool)       : Expr
  | var     (x : String)     : Expr
  | add     (e1 e2 : Expr)   : Expr
  | ifExpr  (cond t f : Expr): Expr
  | lam     (x : String) (body : Expr) : Expr
  | app     (fn arg : Expr)  : Expr
  | letExpr (x : String) (e body : Expr) : Expr

end TypeInfer
```

9.2 The Type Lattice

Rather than types being a simple set, we arrange them in a lattice that captures the degree to which a type is *known*.

Definition — Flat Type Lattice

Define the type information domain:

- $\bot$ (`Unknown`): no type information yet.
- `TInt` , `TBool` : concrete base types.
- `TFun s t` : function type, parameterised by the type information for the domain and codomain (which are themselves in the lattice).
- $\top$ (`TypeError`): contradiction (incompatible constraints).

The order is: $\bot \leq$ everything, everything $\leq \top$, and base types are only $\leq$ themselves (and $\bot/\top$). Function types: `TFun s1 t1 ≤ TFun s2 t2 iff s1 ≤ s2 ∧ t1 ≤ t2` .

```
namespace TypeInfer

inductive Ty where
  | unknown            : Ty    -- ⊥
  | int                : Ty
  | bool               : Ty
  | fun_ (dom cod : Ty): Ty    -- dom → cod
  | typeError          : Ty    -- ⊤

-- Unification of two types: compute their join
def Ty.unify : Ty → Ty → Ty
  | .unknown, t          => t
  | t, .unknown          => t
  | .typeError, _        => .typeError
  | _, .typeError        => .typeError
  | .int, .int           => .int
  | .bool, .bool         => .bool
  | .fun_ d1 c1, .fun_ d2 c2 => .fun_ (Ty.unify d1 d2) (Ty.unify c1 c2)
  | _, _                 => .typeError   -- incompatible types

-- The information order
def Ty.le : Ty → Ty → Prop
  | .unknown, _            => True
  | _, .typeError          => True
  | .int, .int             => True
  | .bool, .bool           => True
  | .fun_ d1 c1, .fun_ d2 c2 => Ty.le d1 d2 ∧ Ty.le c1 c2
  | _, _                   => False

instance : OrderBook.PartialOrder Ty where
  le          := Ty.le
  le_refl     := by intro t; induction t <;> simp [Ty.le, *]
  le_antisymm := by
    intro t1 t2 h12 h21
```

```
    induction t1 <;> cases t2 <;> simp [Ty.le] at * <;>
      try (first | rfl | exact absurd h12 id | exact absurd h21 id)
    case fun_.fun_ d1 c1 ih_d ih_c d2 c2 =>
      have := ih_d h12.1 h21.1; have := ih_c h12.2 h21.2; simp_all
  le_trans    := by
    intro t1 t2 t3 h12 h23
    induction t1 <;> cases t2 <;> cases t3 <;>
      simp [Ty.le] at * <;> try tauto
    case fun_.fun_.fun_ => exact ⟨by tauto, by tauto⟩

instance : Propagators.BoundedJoinSemilattice Ty where
  le          := Ty.le
  le_refl     := OrderBook.PartialOrder.le_refl
  le_antisymm := OrderBook.PartialOrder.le_antisymm
  le_trans    := OrderBook.PartialOrder.le_trans
  bot         := .unknown
  sup         := Ty.unify
  bot_le      := fun _ => trivial
  sup_le_left  := by
    intro a b; induction a <;> cases b <;>
      simp [Ty.le, Ty.unify, *] <;> try exact ⟨by assumption, by assumption⟩
  sup_le_right := by
    intro a b; induction b <;> cases a <;>
      simp [Ty.le, Ty.unify, *] <;> try exact ⟨by assumption, by assumption⟩
  le_sup       := by
    intro a b c hac hbc
    induction a <;> cases b <;> cases c <;>
      simp [Ty.le, Ty.unify] at * <;> tauto

deriving instance DecidableEq for Ty

end TypeInfer
```

9.3 Type Constraint Generation

We generate constraints by walking the expression tree. Each sub-expression gets a fresh *type cell*. Constraints are propagators that merge type information into these cells.

```
namespace TypeInfer

-- A typing context maps variable names to type cells
abbrev Context := List (String × Propagators.Cell Ty)

```

```
def Context.lookup (ctx : Context) (x : String) :
    Option (Propagators.Cell Ty) :=
  ctx.find? (fun p => p.1 == x) |>.map (·.2)

-- Generate type cells and propagators for an expression
-- Returns the type cell for this expression.
partial def inferExpr
    (ctx : Context)
    (e : Expr)
    (steps : IO.Ref (Array Propagators.PropStep)) :
    IO (Propagators.Cell Ty) := do
  match e with
  | .intLit _ =>
    let c ← Propagators.Cell.new (α := Ty)
    let _ ← c.write .int
    return c

  | .boolLit _ =>
    let c ← Propagators.Cell.new (α := Ty)
    let _ ← c.write .bool
    return c

  | .var x =>
    match ctx.lookup x with
    | some c => return c
    | none   =>
      let c ← Propagators.Cell.new (α := Ty)
      let _ ← c.write .typeError  -- unbound variable
      return c

  | .add e1 e2 =>
    let c1 ← inferExpr ctx e1 steps
    let c2 ← inferExpr ctx e2 steps
    let cOut ← Propagators.Cell.new (α := Ty)
    -- Propagator: both inputs and output must be Int
    steps.modify (· .push do
      let _ ← c1.write .int
      let _ ← c2.write .int
      cOut.write .int)
    return cOut

  | .ifExpr cond t f =>
    let ccond ← inferExpr ctx cond steps
    let ct    ← inferExpr ctx t    steps
    let cf    ← inferExpr ctx f    steps
    let cOut  ← Propagators.Cell.new (α := Ty)
```

```
    -- Propagator: cond must be Bool; t and f must agree; out = t
    steps.modify (· .push do
      let _ ← ccond.write .bool
      let tv ← ct.read; let fv ← cf.read
      let _ ← ct.write fv; let _ ← cf.write tv
      let tv' ← ct.read
      cOut.write tv')
    return cOut

  | .lam x body =>
    let cDom  ← Propagators.Cell.new (α := Ty)
    let ctx'  := (x, cDom) :: ctx
    let cBody ← inferExpr ctx' body steps
    let cOut  ← Propagators.Cell.new (α := Ty)
    -- Propagator: out = dom → body
    steps.modify (· .push do
      let d ← cDom.read; let b ← cBody.read
      cOut.write (.fun_ d b))
    return cOut

  | .app fn arg =>
    let cFn  ← inferExpr ctx fn  steps
    let cArg ← inferExpr ctx arg steps
    let cOut ← Propagators.Cell.new (α := Ty)
    -- Propagator: fn must be (arg → out)
    steps.modify (· .push do
      let fv ← cFn.read; let av ← cArg.read
      let ov ← cOut.read
      match fv with
      | .fun_ d r =>
        let _ ← cArg.write d   -- arg type must match domain
        let _ ← cOut.write r   -- result type from codomain
        let _ ← cFn.write (.fun_ av ov) -- refine fn type
        return false            -- (actual change tracked by writes)
      | .unknown =>
        let _ ← cFn.write (.fun_ av ov)
        return false
      | _ =>
        let _ ← cFn.write .typeError; return false)
    return cOut

  | .letExpr x e body =>
    let ce   ← inferExpr ctx  e    steps
    let ctx' := (x, ce) :: ctx
    inferExpr ctx' body steps

```

```
end TypeInfer
```

9.4 Running the Type Inferencer

```
namespace TypeInfer

def infer (e : Expr) : IO Ty := do
  let stepsRef ← IO.mkRef (Array.empty : Array Propagators.PropStep)
  let cResult  ← inferExpr [] e stepsRef
  let steps    ← stepsRef.get
  Propagators.runToFixpoint steps
  cResult.read

-- Helper to show types
def Ty.toString : Ty → String
  | .unknown       => "?"
  | .int           => "Int"
  | .bool          => "Bool"
  | .fun_ d r      => s!"({Ty.toString d} → {Ty.toString r})"
  | .typeError     => "TypeError"

-- Examples
def ex1 : Expr := .add (.intLit 1) (.intLit 2)
-- Expected: Int

def ex2 : Expr := .lam "x" (.add (.var "x") (.intLit 1))
-- Expected: (Int → Int)

def ex3 : Expr := .ifExpr (.boolLit true) (.intLit 1) (.intLit 2)
-- Expected: Int

def ex4 : Expr := .add (.boolLit true) (.intLit 1)
-- Expected: TypeError (Bool ≠ Int)

def runExamples : IO Unit := do
  for (name, e) in [("1 + 2", ex1), ("λx. x+1", ex2),
                    ("if true 1 2", ex3), ("true + 1", ex4)] do
    let ty ← infer e
    IO.println s!"{name} : {Ty.toString ty}"

end TypeInfer
```

Expected output:

```
1 + 2 : Int
λx. x+1 : (Int → Int)
if true 1 2 : Int
true + 1 : TypeError
```

9.5 What the Lattice Buys Us Here

- **Bidirectional propagation is free.** When we call `cArg.write d` inside the `app` propagator, we are propagating type information *backwards*: from the function type to the argument. The lattice join ensures this never loses previously-inferred information.
- **Inconsistency is explicit.** `TypeError` is the $\top$ element. When two incompatible type constraints are merged (`Ty.unify .int .bool = .typeError`), the cell value jumps to $\top$ and stays there. No exception is thrown; the type error propagates naturally through the lattice.
- **Incrementality is free.** If we learn a new fact about x's type (say, x appears in another expression), we just write to x's type cell and re-run the network. The lattice join handles merging old and new constraints correctly.
- **Ordering captures "how much we know".** $\bot$ = `Unknown` $<$ `Int` $<$ `TypeError` = $\top$. The height of a cell's value in the lattice is a measure of how constrained it is. A fully-typed program has all cells at concrete types (neither $\bot$ nor $\top$).

9.6 Exercises

Exercise 9.1 — Let-polymorphism

Lean's type system supports polymorphism. Extend `Ty` with a `TVar (name : String)` constructor and modify `Ty.unify` to do proper unification (substituting type variables). How does this affect the lattice structure?

Exercise 9.2 — Recursive types

Add a `letRec` construct to `Expr` and extend the inferencer to handle it. Note: the type cell for the recursive binding must be initialised to `Unknown` and refined by propagation, enabling inference of e.g. `let rec f x = if x = 0 then 1 else x * f (x-1)`.

Exercise 9.3 — Error messages

When a cell reaches `TypeError`, record *why* (which two types conflicted). Augment the `Ty` type so that `TypeError` carries a `String` explanation, and thread error messages through `Ty.unify`.

Chapter 10

Lean 4 Quick Reference

10.1 Basic Types and Syntax

```
-- Type universes
#check Nat       -- Nat : Type
#check Type      -- Type : Type 1
#check Prop      -- Prop : Sort 0

-- Function types
def f : Nat → Nat → Nat := fun a b => a + b
def g (a b : Nat) : Nat := a + b    -- same, different syntax

-- Dependent types
def myList (α : Type) (n : Nat) : Type := ...

-- Let bindings
def h : Nat :=
  let x := 5
  let y := x + 1
  x * y
```

10.2 Typeclasses

```
-- Declaring a typeclass
class MyClass (α : Type) where
  someMethod : α → Nat
  someProperty : ∀ (a : α), someMethod a ≥ 0

-- Providing an instance
instance : MyClass Nat where
  someMethod := id
  someProperty := Nat.zero_le

```

```
-- Using an instance
def useMyClass [MyClass α] (a : α) : Nat :=
  MyClass.someMethod a
```

10.3 Tactics Reference

Tactic	Usage
`rfl`	Prove $a = a$ by reflexivity
`exact h`	Close goal with proof term `h`
`apply f`	Apply function/lemma `f`
`intro h`	Introduce a hypothesis
`cases h`	Case split on `h`
`induction n`	Induction on `n`
`simp`	Simplify using simp lemmas
`simp\_all`	Simplify using all hypotheses
`omega`	Linear arithmetic over $\mathbb{Z}$
`decide`	Decide propositions by computation
`constructor`	Introduce an And/Iff/...
`have h := ...`	Introduce a local lemma
`tauto`	Tautology checker
`funext`	Function extensionality
`congr`	Congruence

Table 10.1: Commonly used Lean 4 tactics in this book.

10.4 Unicode Input

Symbol	Input	Meaning
≤	`\le`	Less-than-or-equal
≥	`\ge`	Greater-than-or-equal
→	`\to`	Function/implication arrow
←	`\l`	Reverse arrow
↔	`\iff`	If and only if
∀	`\fa`	For all
∃	`\ex`	There exists
⊓	`\sqcap`	Meet (inf)
⊔	`\sqcup`	Join (sup)
⊥	`\bot`	Bottom
⊤	`\top`	Top
∈	`\in`	Set membership
∉	`\notin`	Non-membership
α	`\a`	Greek alpha
β	`\b`	Greek beta
ℕ	`\N`	Natural numbers
∧	`\and`	Logical and
∨	`\or`	Logical or
¬	`\n`	Logical not

Table 10.2: Lean 4 Unicode inputs used in this book.

Selected Solutions

This appendix contains solutions to the exercises that have a single definitive Lean 4 answer. Exercises asking for diagrams, written explanations, or open-ended extensions are left to the reader.

Chapter 1

Exercise 1.1 (Reflexivity and symmetry)

```
theorem symm_antisymm_eq {α : Type} (r : Rel α)
    (hs : Symmetric r) (ha : Antisymm r) :
    ∀ a b : α, r a b → a = b := by
  intro a b h
  -- hs gives us r b a from r a b.
  -- ha then gives a = b from r a b and r b a.
  exact ha a b h (hs a b h)
```

Exercise 1.3 (Product order)

```
instance [OrderBook.PartialOrder α] [OrderBook.PartialOrder β] :
    OrderBook.PartialOrder (α × β) where
  le p q := PartialOrder.le p.1 q.1 ∧ PartialOrder.le p.2 q.2
  le_refl := fun ⟨a, b⟩ =>
    ⟨PartialOrder.le_refl a, PartialOrder.le_refl b⟩
  le_antisymm := fun ⟨a1, b1⟩ ⟨a2, b2⟩ ⟨h1, h2⟩ ⟨h3, h4⟩ => by
    -- Antisymmetry component-wise, then combine with congr
    have ha := PartialOrder.le_antisymm _ _ h1 h3
    have hb := PartialOrder.le_antisymm _ _ h2 h4
    simp_all
  le_trans := fun ⟨a1, b1⟩ ⟨a2, b2⟩ ⟨a3, b3⟩ ⟨h1, h2⟩ ⟨h3, h4⟩ =>
    ⟨PartialOrder.le_trans _ _ _ h1 h3,
     PartialOrder.le_trans _ _ _ h2 h4⟩
```

Chapter 2

Exercise 2.1 (Monotone composition)

Antitone means order-*reversing*: $a \leq b \to f(b) \leq f(a)$. Composing two antitone maps reverses the order twice, yielding a monotone map. A concrete example: negate twice on $\mathbb{Z}$.

```
-- f and g are both antitone (order-reversing) on Int
def f₁ : Int → Int := fun x => -x
def g₁ : Int → Int := fun x => -x

-- Their composition g ∘ f is monotone
example : Monotone (g₁ ∘ f₁) where
  map_le := fun a b h => by
    -- g₁ ∘ f₁ x = -(-x) = x, so it is the identity
    simp [g₁, f₁]
    exact h
```

The general proof that antitone ∘ antitone = monotone:

```
theorem antitone_comp_antitone {α β γ : Type}
    [LE α] [LE β] [LE γ]
    [PartialOrder α] [PartialOrder β] [PartialOrder γ]
    {f : α → β} {g : β → γ}
    (hf : ∀ a b : α, a ≤ b → f b ≤ f a)
    (hg : ∀ a b : β, a ≤ b → g b ≤ g a) :
    Monotone (g ∘ f) where
  map_le := fun a b h =>
    -- a ≤ b →  f b ≤ f a  (hf reverses)
    -- f b ≤ f a →  g (f a) ≤ g (f b)  (hg reverses again)
    hg (f b) (f a) (hf a b h)
```

Exercise 2.2 (Fixed points)

We prove by induction that $f^n(a) \leq f^{n+1}(a)$ for all n, which shows the chain $a \leq f(a) \leq f^2(a) \leq \cdots$ is ascending.

```
-- f^n a means applying f exactly n times to a
def iter {α : Type} (f : α → α) : Nat → α → α
  | 0,     x => x
  | n + 1, x => f (iter f n x)

theorem iter_chain {α : Type} [LE α] [PartialOrder α]
    {f : α → α} (hf : Monotone f)
    {a : α} (ha : a ≤ f a) :
    ∀ n : Nat, iter f n a ≤ iter f (n + 1) a := by
  intro n
  induction n with
```

```
  | zero =>
    -- Base case: iter f 0 a = a ≤ f a = iter f 1 a
    exact ha
  | succ n ih =>
    -- Inductive step: iter f n a ≤ iter f (n+1) a
    -- so f (iter f n a) ≤ f (iter f (n+1) a) by monotonicity
    exact hf.map_le _ _ ih
```

Exercise 2.3 (Identity is monotone)

```
theorem Monotone.id {α : Type} [LE α] [PartialOrder α] :
    Monotone (id : α → α) where
  map_le := fun _ _ h => h
```

The proof is trivial: `id a = a`, so `id a ≤ id b` is just `a ≤ b`, which is exactly the hypothesis.

Chapter 3

Exercise 3.3 (Characterising the order)

```
theorem le_iff_sup_eq {α : Type} [OrderBook.Lattice α] (a b : α) :
    a ≤ b ↔ a ⊔ b = b := by
  constructor
  · intro h
    -- (→) If a ≤ b, show a ⊔ b = b.
    -- a ⊔ b ≤ b because b is an upper bound for both a and b.
    -- b ≤ a ⊔ b by sup_le_right. Antisymmetry closes it.
    apply PartialOrder.le_antisymm
    · exact Lattice.le_sup a b b h (PartialOrder.le_refl b)
    · exact Lattice.sup_le_right a b
  · intro h
    -- (←) If a ⊔ b = b, then a ≤ a ⊔ b = b.
    have : b = a ⊔ b := h.symm
    rw [this]
    exact Lattice.sup_le_left a b
```

Further Reading

- **Davey & Priestley**, *Introduction to Lattices and Order*, 2nd ed., Cambridge University Press, 2002. The standard introductory text on lattice theory, accessible and rigorous.
- **Radul & Sussman**, *The Art of the Propagator*, MIT CSAIL Technical Report, 2009. The foundational paper for the propagator model used in this book.
- **Tate, Stepp, Lerner & Chambers**, *Equality Saturation: a New Approach to Optimization*, POPL 2009. An application of lattice-based fixed-point algorithms to compiler optimization.
- **Nielson, Nielson & Hankin**, *Principles of Program Analysis*, Springer, 1999. Covers dataflow analysis (a lattice-based framework) and abstract interpretation.
- **The Lean 4 Documentation**, `https://lean-lang.org`. Official reference for Lean 4 syntax, tactics, and the core library.
- **Mathlib4**, `https://leanprover-community.github.io/mathlib4_docs/`. The community mathematics library for Lean 4. Once you have finished this book, exploring `Mathlib.Order` will show you how the constructions here scale to a large, verified library.